The {teems} R package user manual

Matthew Cantele

Table of contents

1. Introduction **1**
- 1.1. Motivation . . . 1
- 1.2. Errors and model compatibility . . . 2
- 1.3. About this manual . . . 2

I. Getting started **3**

2. Dependencies **4**
- 2.1. Data . . . 4
- 2.2. R . . . 4
- 2.3. teems-R package . . . 5
- 2.4. teems-solver . . . 5

3. Quickstart **6**
- 3.1. Generate example scripts . . . 6
- 3.2. Step 1: verify the solver with a null run . . . 6
- 3.3. Step 2: run a uniform shock . . . 7
- 3.4. Next steps . . . 7

4. Example files **8**
- 4.1. Overview . . . 8
- 4.2. Arguments . . . 8
- 4.3. Writing model files . . . 9
- 4.4. Writing example scripts . . . 9
- 4.5. Using the generated scripts . . . 11
  - 4.5.1. Script naming convention . . . 11
  - 4.5.2. Structure of a generated script . . . 12
- 4.6. Output . . . 13

II. Data and model inputs **14**

5. Loading and specifying sets **15**
- 5.1. Overview . . . 15
- 5.2. Internal mappings . . . 15
  - 5.2.1. Region (`REG`) . . . 15
  - 5.2.2. Sector (`PROD_COMM`, `ACTS`) . . . 16
  - 5.2.3. Endowment (`ENDW_COMM`, `ENDW`) . . . 16
  - 5.2.4. GTAP-AEZ and GTAP-Power mappings . . . 16
- 5.3. External mappings . . . 17

5.4. Set naming convention . . . 17

6. Loading data **19**
6.1. Overview . . . 19
6.2. Input files . . . 20
6.3. Set mappings . . . 21
6.4. Time steps . . . 21
6.5. Converting formats . . . 21
6.5.1. Loading HAR files as arrays . . . 22
6.5.2. Converting between classic (v6x) and standard (v7x) formats . . . 22
6.5.3. GTAP-AEZ, GTAP-E and GTAP-Power databases . . . 23
6.6. Data aggregation . . . 24
6.6.1. Parameter weights . . . 24
6.6.2. GTAP v6.2 format parameter weight methodology . . . 26
6.6.3. GTAP v7 format parameter weight methodology . . . 27
6.6.4. GTAP-E, GTAP-Power and GTAP-AEZ parameters . . . 30

7. Loading models **31**
7.1. Overview . . . 31
7.2. The model file . . . 31
7.3. Closure selection . . . 32
7.4. Tablo compatibility . . . 33
7.4.1. Supported statements . . . 33
7.4.2. Names . . . 33
7.4.3. Variable types for homogeneity checks . . . 34
7.4.4. Formulas, updates and mappings . . . 34
7.4.5. Sets . . . 34
7.4.6. IF conditions . . . 35
7.5. Condensation . . . 36
7.5.1. Backsolving variables . . . 36
7.5.2. When backsolving pays . . . 37
7.5.3. Condensation statements in the model file . . . 37
7.6. Coefficient value injection . . . 37
7.6.1. Uniform numeric value . . . 37
7.6.2. Data frame . . . 38
7.6.3. CSV file . . . 39
7.6.4. Combining injections . . . 39
7.7. Intertemporal models . . . 39
7.7.1. Leads, lags and parallel solving . . . 40

III. Swaps and shocks **41**

8. Closure swaps **42**
8.1. Overview . . . 42
8.2. Simple (full variable) swaps . . . 42
8.2.1. Choosing swaps . . . 43

8.3. Complex (partial variable) swaps . . . 43
8.3.1. ems_swap() . . . 43
8.3.2. Subset swaps . . . 45

9. Uniform shocks **47**
9.1. Overview . . . 47
9.2. Full variable uniform shocks . . . 47
9.3. Partial variable uniform shocks . . . 48
9.4. Subset shocks . . . 48
9.5. Multiple shocks and mixed swaps . . . 49

10. Custom shocks **50**
10.1. Overview . . . 50
10.2. Full variable custom shocks . . . 50
10.3. CSV input . . . 52
10.4. Partial variable custom shocks . . . 53
10.5. Chronological year input . . . 53

11. Scenario shocks **55**
11.1. Overview . . . 55
11.2. Using scenario shocks . . . 55
11.3. Population trajectory example . . . 56
11.4. Partial scenario shocks . . . 58

IV. Deploying and solving the model **59**

12. Deploying the model **60**
12.1. Overview . . . 60
12.2. Arguments . . . 60
12.3. Basic usage . . . 61
12.4. Deploying with shocks and swaps . . . 61
12.5. Closure swaps . . . 62
12.6. User-generated inputs . . . 63

13. Solving the model **64**
13.1. Overview . . . 64
13.2. Basic usage . . . 64
13.3. Arguments . . . 65
13.4. Solution methods . . . 66
13.4.1. Step counts and single-pass runs . . . 67
13.4.2. Runge-Kutta controls . . . 67
13.5. Matrix methods . . . 69
13.5.1. Choosing a method and resources . . . 70
13.6. Memory . . . 74
13.7. Checking solve quality . . . 75
13.8. Reproducibility . . . 75
13.9. Solver version . . . 75
13.10. Concurrent runs . . . 76

13.11 Terminal mode . . . 76
13.12 Solving in-situ . . . 76
13.12.1 Arguments . . . 77
13.12.2 Input files . . . 78

14. Composing results **79**
14.1. Overview . . . 79
14.2. Arguments . . . 79
14.3. Basic usage . . . 80
14.4. Filtering output . . . 80
14.5. Return value . . . 80
14.6. Levels results . . . 81
14.7. Separate solutions . . . 81

V. Configuration **83**

15. Package options **84**
15.1. Overview . . . 84
15.2. Retrieving options . . . 85
15.3. Setting options . . . 85
15.4. Resetting options . . . 85
15.5. Available options . . . 85

Appendices **88**

A. Solver benchmarks **88**
A.1. How the runs were made . . . 88
A.2. Intertemporal models . . . 88
A.3. Static models, full system . . . 89
A.4. Static models, condensed . . . 89
A.5. Large models . . . 89
A.6. Memory . . . 90

B. References **91**
B.1. Tablo language and solution procedure . . . 91
B.2. Complementarities and subtotals . . . 91
B.3. Runge-Kutta methods . . . 91
B.4. Parallel matrix methods . . . 92
B.5. Sparse linear algebra . . . 92
B.6. Software . . . 92
B.7. Models . . . 93

1. Introduction

Reproducibility within computable general equilibrium (CGE) and partial equilibrium modeling analyses has long been elusive, in part due to proprietary data and software. Here we address one aspect of this equation with the introduction of the TEEMS (Trade and Environment Equilibrium Modeling System) open-source software suite.

At its core, TEEMS comprises an R package `teems` used to compose model runs and teems-solver, a C-based solver capable of solving a large system of nonlinear equations by leveraging a range of mathematical and scientific computation libraries. Auxiliary repositories to facilitate model runs include teems-models, containing internally available and immediately compatible models, and teems-mappings, which contains ready-to-use mappings from coarse to fine resolutions for all core sets in most GTAP-based models.

With a base within the general purpose R scripting language, users are able to seamlessly adjust model components including set aggregations, parameter weighting, closure and shock specifications as well as a large number of options pertaining to the solution and model outputs. Model scripts can then be shared with colleagues, reviewers, and policy-makers for vetting, modification, and reproduction.

1.1. Motivation

Reproducibility has been elusive also due to the sheer complexity of compiling a single model run. A "simple" model run, for example, *broadly* involves:

1. Data and set selection
2. Set mappings
3. Model selection
4. Condensation (backsolving)
5. Closure selection including any swaps
6. Shock designation
7. Set-specific basedata loading, modification, and aggregation
8. Set- and weight-specific parameter loading, modification, and aggregation
9. Selection of matrix solution methods
10. Number of step and subinterval designations
11. Solution of the nonlinear equation system
12. Composition of all outputs into structured data

Due to the complexity involved, in the course of building the TEEMS framework we found that in the absence of a reproducible architecture, there is a very high possibility of persistent logic errors propagating through to the output without any obvious indications that this is taking place.

Models which incorporate time steps and additional auxiliary data (e.g., land-use data) or are coupled with other models as is the case with most integrated assessment models (IAMs) are even more complex and thus more susceptible to a wide range of errors.

Ultimately we hope that this software mitigates the steep learning curve requisite to running equilibrium models and allows scientists from various disciplines to advance model representation of their fields. The TL;DR – running a CGE model *does not* have to be so hard!

1.2. Errors and model compatibility

This software suite has been under development for many years. Despite our best efforts, it is possible that you will encounter issues – particularly given the wide range of Tablo model files that exist in the field. If your model file does not work, please get in touch and we will seek to expand compatibility.

1.3. About this manual

This manual is a step-by-step user guide to `teems`.

Part I.

Getting started

2. Dependencies

This chapter outlines what is needed to run your first CGE model. Ideally all components would be open-source and perhaps one day they will be, but for the moment there are still some minor hurdles to overcome. Before beginning to write model scripts in R with `teems`, users will need to gain access to a GTAP database and build the teems-solver. GEMPack users can skip the solver step although they will be missing out on a considerable degree of functionality both within the solution process itself and in post-model data availability.

For those skeptical of using R or under the impression that R is only for statistical analysis, we would invite you to investigate speed tests between the R data.table package (widely employed within `teems`) and comparable packages in Python and Julia. The teems-solver is written in C and uses highly performant Fortran libraries.

2.1. Data

The only data currently teems-compatible is available from the Global Trade Analysis Project (GTAP). Although the most recent databases are proprietary, GTAP has consistently made databases available two versions behind the current release: with GTAP 12 current, GTAP 10 is free.

To access the freely available database, users will need to register with GTAP and download the "FlexAgg" format. For GTAP 10 this is accessible under the "GDyn Data Base" entry in the GTAP 10 "Satellite Data and Utilities" section here. For GTAP members that have purchased a database, the current `teems` version is capable of handling the "FlexAgg" format for GTAP Databases 9 through 12. Models requiring additional headers (e.g., GTAP-EP) require the associated FlexAgg equivalent databases.

2.2. R

R (>= 4.3.0) is required. Download and install the latest release for your platform from CRAN.

RStudio is the recommended IDE for working with `teems` scripts. It provides a script editor, integrated console, and a file browser that makes it straightforward to open, edit, and run the example scripts generated by `ems_example()`. The 49th Parallel theme with Fira Code font makes for an aesthetically pleasing coding experience.

2.3. teems-R package

The R package is available on CRAN.

```
# install package
install.packages("teems")
```

2.4. teems-solver

`teems` calls Docker to solve the model's nonlinear equation system using low-level C and Fortran routines. The solver build is available here: teems-solver. The latest solver build is always compatible with the latest R package version. `teems` 1.0.0 requires teems-solver 1.1.0 or later and checks the image's version before every solve: an image built for an earlier release stops with a rebuild message. Rebuild it as described in the teems-solver README ("Updating an Existing Installation") and restart R.

Two prerequisites:

- **Docker** — install from docker.com. Linux users must also configure Docker to run without `sudo`.
- **HSL libraries** — the sparse linear algebra packages the solver uses are obtained as a single **libHSL** source snapshot (`libHSL.v2026.8.4.tar.gz`) from the STFC licensing portal prior to building. These are available at no cost for academic use.

Two Docker build approaches are available. The **expedited build** (~5 min) is recommended for most users and uses a pre-built base image containing all open-source dependencies. The **full build** (~40 min) compiles all dependencies from source.

3. Quickstart

This chapter gets you from a fresh install to a solved CGE model in as few steps as possible, using the freely available GTAP 10 database and the classic GTAP model (version 6.2).

Before continuing, make sure you have completed all prerequisites in the Dependencies chapter:

- R (>= 4.3.0) installed
- `teems` R package installed
- GTAP 10 FlexAgg HAR files downloaded
- `teems-solver` Docker image built

3.1. Generate example scripts

`ems_example()` writes a set of ready-to-run R scripts to disk, with your data paths already injected at the top of each file. Point it at your HAR files and a project directory:

```
library(teems)

paths <- ems_example(
  model = "GTAPv6",
  path = "~/my_project",
  type = "scripts",
  dat_input = "v6_data/gsddat.har",
  par_input = "v6_data/gsdpar.har",
  set_input = "v6_data/gsdset.har"
)
```

`paths` (the object returned above) is a character vector of file paths to R scripts that are self-contained and immediately runnable.

3.2. Step 1: verify the solver with a null run

Run `null.R` first. A null run applies no shocks, so every model variable should return zero. Note that the examples below use `source()` to run a specific script but for interactive runs opening the scripts generated by `ems_example()` within RStudio is recommended.

```
source(paths[basename(paths) == "null.R"])

# null.R performs these checks automatically:
check        # TRUE  - all variable values are 0
var_check    # TRUE  - output contains the expected number of variables
coeff_check  # TRUE  - output contains the expected number of coefficients
```

3.3. Step 2: run a uniform shock

`full_uniform.R` applies a 1% population shock (variable: `pop`) uniformly across all regions.

```
source(paths[basename(paths) == "full_uniform.R"])

# full_uniform.R checks that the shock came through:
check  # TRUE - all pop values equal 1 (percentage change)
```

`outputs` is a tibble where each row corresponds to a solved variable or coefficient. The `dat` list column holds the result data for that row in data.table format.

```
# Inspect results for one variable
outputs$dat$pop
```

See the Composing results chapter for a full guide to working with `outputs`.

3.4. Next steps

The other scripts in `paths` cover the full range of shock and swap configurations available for GTAPv6. Recommended progression:

Script	Try it when…
`part_uniform.R`	You want to shock a single region or sector
`part_uniform_part_swap.R`	You need to change the closure for part of the model
`custom_full.R`	You have heterogeneous shock values from external data
`full_uniform.R` with a different variable	You want to explore other exogenous variables

For a deeper look at each script pattern and how to customize them, see the Example files chapter. For data aggregation options (region and sector mappings), see Loading and specifying sets.

4. Example files

4.1. Overview

`ems_example()` writes bundled example model and script files to disk. It is a good starting point for new users, providing ready-to-run material for each of the six vetted models without requiring any additional setup beyond GTAP data access.

Two output modes are available, controlled by the `type` argument:

- `"model_files"` (default): writes the model (`.tab`) and closure (`.cls`) files for the chosen model.
- `"scripts"`: writes a complete set of example R scripts covering a range of shock, swap, and solver configurations supported by that model. Each script is a self-contained, runnable workflow from data loading through to result parsing.

```
ems_example(
  model,
  path,
  type = c("model_files", "scripts"),
  dat_input = NULL,
  par_input = NULL,
  set_input = NULL
)
```

4.2. Arguments

Argument	Default	Description
`model`		Model name. One of `"GTAPv6"`, `"GTAPv7"`, `"GTAP-RE"`, `"GTAP-AEZ"`, `"GTAP-E"`, `"GTAP-EP"`
`path`		Directory where files will be written
`type`	`"model_files"`	Output type: `"model_files"` or `"scripts"`
`dat_input`	`NULL`	Path to the basedata HAR file, or the `dat` list from `GTAP_convert()`. Required when `type = "scripts"`

Argument	Default	Description
`par_input`	`NULL`	Path to the parameters HAR file, or the `par` list from `GTAP_convert()`. Required when `type = "scripts"`
`set_input`	`NULL`	Path to the sets HAR file, or the `set` list from `GTAP_convert()`. Required when `type = "scripts"`

4.3. Writing model files

The simplest use of `ems_example()` extracts a model's Tablo and closure files so they can be inspected or used directly with `ems_model()` or `solve_in_situ()`.

```
library(teems)

# Write GTAPv7 model files to a temporary directory
paths <- ems_example("GTAPv7", tempdir())

# Or to a specific directory
paths <- ems_example("GTAP-RE", "~/my_project/model")
```

The returned list contains two named paths:

```
paths[["model_file"]]   # path to the .tab file
paths[["closure_file"]] # path to the .cls file
```

These can be passed directly to `ems_model()`:

```
model <- ems_model(
  model_file = paths[["model_file"]],
  closure_file = paths[["closure_file"]]
)
```

4.4. Writing example scripts

When `type = "scripts"`, `ems_example()` writes a directory of R scripts covering a range of configurations available for the chosen model. Each script is injected with the supplied data paths and is immediately runnable.

```
paths <- ems_example(
  model = "GTAPv7",
  path = "~/my_project",
  type = "scripts",
  dat_input = "~/dat/GTAP/v12/flexAgg/gsdfdat.har",
  par_input = "~/dat/GTAP/v12/flexAgg/gsdfpar.har",
  set_input = "~/dat/GTAP/v12/flexAgg/gsdfset.har"
)
```

The GTAP-AEZ scripts need a GTAP-AEZ database (GTAP 11c or newer). The scripts pass the HAR files to `ems_data()`, which prepares the AEZ layer on load; `GTAP_convert(target = "GTAP-AEZ")` gives the prepared, unaggregated database in R (see GTAP-AEZ, GTAP-E and GTAP-Power databases):

```
paths <- ems_example(
  model = "GTAP-AEZ",
  path = "~/my_project",
  type = "scripts",
  dat_input = "~/dat/GTAP-AEZ/v12a/gsdfdat.har",
  par_input = "~/dat/GTAP-AEZ/v12a/gsdfpar.har",
  set_input = "~/dat/GTAP-AEZ/v12a/gsdfset.har"
)
```

The GTAP-E scripts need a GTAP-E database (GTAP 12a or newer), prepared by `ems_data()` on load or with `GTAP_convert(target = "GTAP-E")` (see GTAP-AEZ, GTAP-E and GTAP-Power databases). They aggregate with the `energy` mapping, which keeps the energy commodities distinct as the model's nest requires:

```
paths <- ems_example(
  model = "GTAP-E",
  path = "~/my_project",
  type = "scripts",
  dat_input = "~/dat/GTAP-E/12a/gsdfdat.har",
  par_input = "~/dat/GTAP-E/12a/gsdfpar.har",
  set_input = "~/dat/GTAP-E/12a/gsdfset.har"
)
```

The GTAP-EP scripts need a GTAP-Power database (GTAP 12a or newer), prepared by `ems_data()` on load or with `GTAP_convert(target = "GTAP-EP")` (see GTAP-AEZ, GTAP-E and GTAP-Power databases). They aggregate with the `power` mapping, which keeps transmission, base load and peak load apart as the generation nest requires:

```
paths <- ems_example(
  model = "GTAP-EP",
  path = "~/my_project",
```

```
  type = "scripts",
  dat_input = "~/dat/GTAP-P/12a/gsdfdat.har",
  par_input = "~/dat/GTAP-P/12a/gsdfpar.har",
  set_input = "~/dat/GTAP-P/12a/gsdfset.har"
)
```

The data inputs can also be the in-memory list objects returned by `GTAP_convert()`, which is useful when the raw data needs format conversion before running:

```
# Convert v7 data to v6.2 format, then generate scripts for the classic model
converted <- GTAP_convert(
  dat_har = "~/dat/GTAP/v12/gsdfdat.har",
  par_har = "~/dat/GTAP/v12/gsdfpar.har",
  set_har = "~/dat/GTAP/v12/gsdfset.har",
  target = "GTAPv6"
)

paths <- ems_example(
  model = "GTAPv6",
  path = "~/my_project",
  type = "scripts",
  dat_input = converted$dat,
  par_input = converted$par,
  set_input = converted$set
)
```

When list objects are supplied, `ems_example()` writes data to RDS files alongside the scripts and injects `readRDS()` calls at the top of each script in place of plain path assignments.

4.5. Using the generated scripts

`ems_example()` returns a character vector of paths, one per written script. Each script is ready to run; the data paths supplied to `ems_example()` are injected at the top of the file as plain assignments.

4.5.1. Script naming convention

Script file names encode the shock and swap configuration (the exact set varies by model):

File name	What it demonstrates
`null.R`	Zero-shock run — useful for verifying the model solves before applying shocks

File name	What it demonstrates
`numeraire.R`	Uniform shock to the numeraire (`pfactwld`): all prices move by the shock and real variables stay unchanged
`full_uniform.R`	Uniform shock across all elements of a variable
`part_uniform.R`	Uniform shock on a subset of elements
`part_uniform_full_swap.R`	Partial shock plus a full-variable closure swap
`part_uniform_part_swap.R`	Partial shock plus a partial closure swap
`part_uniform_part_swap_mixed.R`	Partial shock plus mixed (full + partial) swaps
`part_uniform_subset_swap.R`	Partial shock and swaps with mixed multiple elements and subsets
`custom_full.R`	Heterogeneous shock values supplied as a data frame
`custom_partial.R`	Heterogeneous shock values supplied as a data frame on a subset of elements
`custom_full_csv.R`	Heterogeneous shock values read from a CSV file
`custom_partial_*.R`	Heterogeneous shocks with one or more dimensions fixed to elements
`*_year.R`, `scenario.R`	Intertemporal models only: shocks by chronological year, and a scenario shock

4.5.2. Structure of a generated script

Each script begins with the injected assignments followed by the template body. Opening `part_uniform.R` in an editor shows:

```
dat_input = "~/dat/GTAP/v12/flexAgg/gsdfdat.har"
par_input = "~/dat/GTAP/v12/flexAgg/gsdfpar.har"
set_input = "~/dat/GTAP/v12/flexAgg/gsdfset.har"
model_file = "~/my_project/GTAP-RE.tab"
closure_file = "~/my_project/GTAP-RE.cls"

dat <- ems_data(
  dat_input = dat_input,
  par_input = par_input,
  set_input = set_input,
  time_steps = c(0, 1, 2),
  REG = "big3",
  ACTS = "macro_sector",
  ENDW = "labor_agg"
)

model <- ems_model(
  model_file = model_file,
  closure_file = closure_file
)
```

```
partial <- ems_uniform_shock(
  var = "aoall",
  value = -1,
  REGr = "chn",
  ACTSa = "crops"
)

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = partial
)

outputs <- ems_solve(
  cmf_path = cmf_path,
  matrix_method = "SBBD",
  n_tasks = 2L,
  solution_method = "Gragg"
)
```

4.6. Output

`ems_example()` returns a character vector of file paths to the written files. For `type = "model_files"` this is the `.tab` and `.cls` paths. For `type = "scripts"` this is one path per written script.

Part II.

Data and model inputs

5. Loading and specifying sets

5.1. Overview

Set mappings aggregate the granular regions, sectors, and endowments in the GTAP database to the resolution desired for a given model run. In `ems_data()`, these are passed as named arguments via `...`, where each argument name corresponds to a set name in the model Tablo file.

Any set provided with a mapping is treated as a **parent mapping** and aggregated accordingly. Sets not explicitly mapped are handled automatically: if a set's elements are a subset of a mapped set's elements, the parent mapping is inherited; if a set's elements span multiple mapped sets, the relevant portion of each parent mapping is applied. If a set has no element-level relationship to any mapped set, it remains unaggregated.

For the base GTAP models the minimum required mappings are:

Format	Minimum mappings
GTAPv6.2	`REG`, `PROD_COMM`, `ENDW_COMM`
GTAPv7	`REG`, `ACTS`, `ENDW`

All other read-in sets (e.g. `MARG_COMM` in GTAPv6.2, `MARG` in GTAPv7) are derived automatically from the parent mapping. Additional mappings can always be supplied explicitly to override the inferred result.

5.2. Internal mappings

Internal mappings ship with `teems` and are specified by a string name without a `.csv` extension. All internal mappings, organized by database version, data format, and set name, are available at teems-mappings.

5.2.1. Region (`REG`)

Region mappings apply to the `REG` set in both GTAPv6.2 and GTAPv7 formats. The `AR5`, `WB7`, and `WB23` mappings are derived from the countrycode R package (ar5, region, and region23 respectively).

Name	Description
`big3`	China (`chn`), United States (`usa`), Rest of World (`row`)

Name	Description
AR5	IPCC Fifth Assessment Report regions: asia, eit, lam, maf, oecd
WB7	World Bank 7-region classification
WB23	World Bank 23-region classification
R32	IIASA-based SSP 32-region mapping
medium	Custom medium-resolution aggregation
large	Custom large-resolution aggregation
huge	Near country-level with minor groupings
full	Full (unaggregated)

5.2.2. Sector (PROD_COMM, ACTS)

Sector mappings apply to PROD_COMM in GTAPv6.2 and ACTS in GTAPv7.

Name	Description
macro_sector	Broad evenly weighted aggregation: crops, food, livestock, mnfcs, svces
food	Food-focused: disaggregated food and agriculture; aggregated manufacturing and services
energy	Energy-focused: disaggregated energy sectors
energy_nest	Coal, gas, oil and electricity kept apart over a macro remainder; the coarsest aggregation GTAP-E admits
manufacturing	Manufacturing-focused: disaggregated manufacturing sectors
services	Services-focused: disaggregated services sectors
medium	Custom medium-resolution aggregation
full	Full (unaggregated)

5.2.3. Endowment (ENDW_COMM, ENDW)

Endowment mappings apply to ENDW_COMM in GTAPv6.2 and ENDW in GTAPv7.

Name	Description
labor_agg	Aggregated labor: capital, labor, land, natlres
labor_diff	Differentiated labor: capital, land, natlres, sklab, unsklab
full	Full (unaggregated)

5.2.4. GTAP-AEZ and GTAP-Power mappings

GTAP-AEZ and GTAP-Power databases redefine some sets, so they have mappings of their own; every other set uses the mappings above.

The 18 AEZs cross six length-of-growing-period bands with three climate zones (Lee et al. 2005; see References).

Set	Model	Name	Description
AEZS, ENDW	GTAP-AEZ	lgp	The 18 AEZs pooled into six length-of-growing-period bands (other endowments unchanged)
AEZS, ENDW	GTAP-AEZ	climate	The 18 AEZs pooled into three climate zones: boreal, temperate, tropical
ACTS	GTAP-EP	power	The GTAP-Power reference aggregation: transmission, base load and peak load apart
ACTS	GTAP-EP	power_tech	Every generation technology kept apart

5.3. External mappings

Custom mappings can be provided as a file path to a two-column CSV or a data frame (e.g., `data.table` or `tibble`), where the first column contains origin elements and the second column contains destination (aggregated) elements. See teems-mappings for example files. Set element names are case-insensitive on input and lowercase everywhere else: the elements a database ships (`Land`, `NatlRes`, `CGDS`), the origin and destination columns of a mapping, and the elements named in shocks, swaps and closures are all folded to lowercase, and every TEEMS object and solver output carries them in lowercase.

```
v7_data <- ems_data(dat_input = "v7_data/gsdfdat.har",
                    par_input = "v7_data/gsdfpar.har",
                    set_input = "v7_data/gsdfset.har",
                    REG = "path/to/custom_REG_mapping.csv",  # external mapping
                    ACTS = "macro_sector",
                    ENDW = "labor_agg"
)
```

5.4. Set naming convention

Set names in ***model-specific context*** (e.g., shock functions, data loaded directly into `ems_model()`) must be provided as the model-specific concatenation of the standard set name plus the variable-specific index. For example:

Set	Index	Column name
REG	r	REGr
REG	s	REGs
COMM	c	COMMc
ACTS	a	ACTSa
ENDW	e	ENDWe

Set	Index	Column name
ALLTIME	t	`ALLTIMEt`

This naming convention disambiguates data entries for variables that have multiple instances of the same set. These names are case-insensitive: `regr` and `REGR` both resolve to `REGr`. Set position (i.e., column order) is inconsequential for any inputs.

6. Loading data

6.1. Overview

The `ems_data()` function loads and prepares GTAP database files for use in CGE model runs. It handles the three core GTAP data files (dat, par, and set) and is also used to specify any time steps (for temporally dynamic models) and load set mappings for sets that are read into the model (i.e., not constructed from set operations).

```
ems_data(
  dat_input,
  par_input,
  set_input,
  time_steps = NULL,
  par_weights = "value",
  ...
)
```

Argument	Type	Description
`dat_input`	Character or list	Path to a HAR file containing basedata coefficient data, or a `dat` list from `GTAP_convert()`
`par_input`	Character or list	Path to a HAR file containing parameter coefficient data, or a `par` list from `GTAP_convert()`
`set_input`	Character or list	Path to a HAR file containing set elements and attributes, or a `set` list from `GTAP_convert()`
`time_steps`	Integer vector or `NULL`	Time steps for intertemporal models (see Time steps)
`par_weights`	Character	Weighting of parameters under aggregation, `"value"` (default) or `"share"`, for all parameters or per header (see Parameter weights)
`...`	Named arguments	Set mappings for read-in sets (see Loading and specifying sets)

GTAP databases need all three input files (`dat_input`, `par_input`, `set_input`) and the model-specific set mappings; a database held in a single HAR file needs `dat_input` alone (see below). `time_steps` is required for intertemporal models.

6.2. Input files

Compatible input files are currently limited to those produced by the Global Trade Analysis Project (GTAP). Although the most recent database releases are proprietary, GTAP has consistently made databases publicly available two versions behind the current release: with GTAP 12 current, GTAP 10 is free.

In order to access the freely available database, users will need to register with GTAP and download the "FlexAgg" format. For GTAP 10 this is accessible under the "GDyn Data Base" entry in the GTAP 10 "Satellite Data and Utilities" section here. For paying GTAP members, the current `teems` version is capable of handling the "FlexAgg" format for GTAP Databases 9 through 12.

The following command will load data consistent with the standard GTAP model, using broad aggregations for regions, commodities, and endowments. Users will need to specify the location of data (`"v7_data/gsdfdat.har"` placeholder).

```
v7_data <- ems_data(dat_input = "v7_data/gsdfdat.har", # basedata coefficients
                    par_input = "v7_data/gsdfpar.har", # parameter coefficients
                    set_input = "v7_data/gsdfset.har", # set elements
                    REG = "big3",
                    ACTS = "macro_sector",
                    ENDW = "labor_diff"
)
```

Note that the actual names of HAR data files vary according to the GTAP release.

A database held in a single HAR file, such as the ORANI-G `basedata.har`, is loaded by giving `dat_input` alone. Every character header with distinct entries becomes a set named after its header, every real header a coefficient dimensioned by the set labels stored in the file, and the data are passed through at full resolution; set mappings do not apply on this route.

```
orani_data <- ems_data(dat_input = "oranig/basedata.har")
```

Sets read this way keep the element order of their header in the file; so does any set passed through without aggregation on the GTAP route. Aggregated sets are ordered by their mapping. The order matters to a model that works by position, for example through `$POS` or a mapping read from a header.

A real header stored without set labels is laid out over the sets declared by the coefficient that reads it, in the file's storage order. If the header's size or shape does not fit that declaration, `ems_deploy()` stops with an error naming the header and the coefficient.

Input Type	GTAP v9	GTAP v10	GTAP v11	GTAP v12
dat_input	gddat.har	gsddat.har	gsdfdat.har	gsdfdat.har
par_input	gdpar.har	gsdpar.har	gsdfpar.har	gsdfpar.har
set_input	gdset.har	gsdset.har	gsdfset.har	gsdfset.har

6.3. Set mappings

Set mappings aggregate the fine-grained regions, sectors, and endowments in the GTAP database down to the resolution required for a given model run. The set names used in `...` depend on the data format — see the Loading and specifying sets chapter for the full list of internal mappings and instructions for supplying custom CSV mappings.

6.4. Time steps

For temporally dynamic models (e.g., GTAP-RE), time steps must be provided representing t0 plus actual year steps from t0. Time steps can be inputted in either actual year increments or represented as chronological years. Note that if chronological years are used, t0 must correspond with the reference year of the database being used.

The following uses explicit time steps (steps from t0), equivalent to chronological years `c(2017, 2018, 2019, 2020, 2021, 2023, 2025, 2027, 2029, 2031, 2033)` for a database with a 2017 reference year.

```
data <- ems_data(dat_input = "v7_data/gsdfdat.har",
                 par_input = "v7_data/gsdfpar.har",
                 set_input = "v7_data/gsdfset.har",
                 time_steps = c(0, 1, 2, 3, 4, 6, 8, 10, 12, 14, 16),
                 REG = "WB23",
                 ACTS = "services",
                 ENDW = "labor_agg"
)
```

Chronological time steps (with t0 corresponding to the reference year of input data).

```
data <- ems_data(dat_input = "v7_data/gsdfdat.har",
                 par_input = "v7_data/gsdfpar.har",
                 set_input = "v7_data/gsdfset.har",
                 time_steps = c(2017, 2018, 2020, 2022, 2024, 2026, 2028, 2030),
                 REG = "R32",
                 ACTS = "medium",
                 ENDW = "labor_diff"
)
```

6.5. Converting formats

`GTAP_convert()` reads a GTAP database's HAR files into R at full resolution, without any aggregation, which makes it the easy way to inspect or modify a database. With `target`, it can also convert a standard GTAP database between the v6.2 and v7 formats, or prepare a GTAP-AEZ, GTAP-E or GTAP-Power database for its model. The returned list can be passed directly to `ems_data()`

or `ems_example()`. After a conversion, set mappings are specified for the target format, *not* the original format of the data.

```
GTAP_convert(
  dat_har,
  par_har,
  set_har,
  target = NULL
)
```

Argument	Type	Description
`dat_har`	Character	Path to the basedata HAR file
`par_har`	Character	Path to the parameter HAR file
`set_har`	Character	Path to the set HAR file
`target`	Character or `NULL`	`NULL` reads the files as they are. `"GTAPv6"` or `"GTAPv7"` converts a standard GTAP database to that format (standard GTAP databases only). `"GTAP-AEZ"`, `"GTAP-E"` or `"GTAP-EP"` prepares that model's database, without converting its format

6.5.1. Loading HAR files as arrays

When `target` is omitted, `GTAP_convert()` simply parses the HAR files into lists of arrays:

```
dat <- GTAP_convert(
  dat_har = "v7_data/gsdfdat.har",
  par_har = "v7_data/gsdfpar.har",
  set_har = "v7_data/gsdfset.har"
)

v7_data <- ems_data(dat_input = dat$dat,
                    par_input = dat$par,
                    set_input = dat$set,
                    REG = "big3",
                    ACTS = "macro_sector",
                    ENDW = "labor_diff"
)
```

6.5.2. Converting between classic (v6x) and standard (v7x) formats

To use the classic model with a v7-format database, or the standard model with a v6-format database, specify `target`. Format conversion applies to the standard GTAP database only.

A GTAP 12 database converted to v6.2 format for the classic GTAP model:

```
converted <- GTAP_convert(
  dat_har = "v7_data/gsdfdat.har",
  par_har = "v7_data/gsdfpar.har",
  set_har = "v7_data/gsdfset.har",
  target = "GTAPv6"
)

v6_data <- ems_data(dat_input = converted$dat,
                    par_input = converted$par,
                    set_input = converted$set,
                    REG = "big3",
                    PROD_COMM = "macro_sector",
                    ENDW_COMM = "labor_agg"
)
```

A GTAP 10 database converted to v7 format for the standard GTAP model:

```
converted <- GTAP_convert(
  dat_har = "v6_data/gsddat.har",
  par_har = "v6_data/gsdpar.har",
  set_har = "v6_data/gsdset.har",
  target = "GTAPv7"
)

v7_data <- ems_data(dat_input = converted$dat,
                    par_input = converted$par,
                    set_input = converted$set,
                    REG = "AR5",
                    ACTS = "macro_sector",
                    ENDW = "labor_agg"
)
```

6.5.3. GTAP-AEZ, GTAP-E and GTAP-Power databases

The GTAP-AEZ, GTAP-E and GTAP-EP models read sets and parameters that their databases do not ship in the form the model expects. `ems_data()` recognises these databases and prepares them on load, so the HAR files can be passed directly. To work with a prepared database in R before aggregation, use `GTAP_convert()` with `target = "GTAP-AEZ"`, `"GTAP-E"` or `"GTAP-EP"`; this prepares the database (generates new headers) but does not convert its format. Conversion between v6 and v7 versions of these databases is not supported.

```
e_data <- ems_data(dat_input = "e_data/gsdfdat.har",
                   par_input = "e_data/gsdfpar.har",
                   set_input = "e_data/gsdfset.har",
                   REG = "big3",
```

```
                    ACTS = "energy"
)
```

6.6. Data aggregation

Data is aggregated according to type, with non-parameter coefficients summed by target set mappings and weighted averages calculated for the parameters below. A simple mean is applied to parameters not listed below, and to the two base data headers that are not values: `DPSM` (the sum of the household distribution parameters, 1 in every region) and, in GTAP-E and GTAP-Power data, `GSHR` (the government share of carbon tax revenue). If custom parameter values are desired, these can be directly loaded in their final format into the `ems_model()` function, and no aggregation or modification will take place. In the below notation, hat (ˆ) indicates the new parameter value and asterisk indicates the destination set mapping.

6.6.1. Parameter weights

The elasticities are aggregated region by region under the share method, so each aggregated region keeps its own values; the value method follows FlexAgg, which weights the Armington, intermediate-input and value-added elasticities (ESBD, ESBM, ESDC, ESMC, ESBC, ESBV) with world totals and writes one value to every region. Two weighting methods are available through the `par_weights` argument of `ems_data()`:

- `"value"` (default): each member is weighted by the value of its nest, as the GTAP aggregation programs (FlexAgg) do; the formulas are given below. The region-generic parameters (ESBD, ESBM, ESDC, ESMC, ESBC, ESBV) are weighted with world totals and take one value in every region, as FlexAgg writes them; a parameter file that carries them region-specific is first averaged over regions with the same weights.
- `"share"`: each member is weighted by the value of its CES or CET nest, V, times one minus the sum of its squared input shares θ_k,

$$w = V\left(1 - \sum_k \theta_k^2\right).$$

  This is the weight under which the aggregate elasticity reproduces the members' response to a small change in relative prices. A member whose input shares cannot move carries little weight: a commodity that is almost never imported cannot shift the aggregate import share, whatever its elasticity, and neither can imports bought almost entirely from one source. For a nest with two inputs the weight reduces to $2V\theta(1-\theta)$. Shares are taken over the inputs of the aggregated model (for import sourcing, the aggregated source regions). Where every member of an aggregate has a zero weight, the value weight is used instead; where that is zero too (an activity or endowment a region does not have), the members are averaged. Where such members dominate an aggregate, as oil and gas do in the mining sector of an importing region, the share-weighted elasticity can fall far below the value-weighted one.

A single method applies to every weighted parameter; individual parameters can be set apart by name:

```
v7_data <- ems_data("v7_data/gsdfdat.har",
                    "v7_data/gsdfpar.har",
                    "v7_data/gsdfset.har",
                    REG = "big3",
                    ACTS = "macro_sector",
                    ENDW = "labor_agg",
                    par_weights = c(ESBM = "share"))
```

The CDE parameters (INCP, SUBP) are not substitution elasticities and are weighted by private consumption under either method. The nests behind the share weights:

Header	Nest	Inputs (θ_k)	Value (V)
ESBD	domestic/imported, one per agent	domestic, imported	purchases by firms, households, government and investment (v6: firms, households, government)
ESBM	import sourcing	aggregated source regions	bilateral imports (VMSB; v6 VIMS)
ESDC, ESMC	the v12 CEPII-based Armington set (unread by the shipped models)	as ESBD, ESBM	as ESBD, ESBM
ESBV	primary factors	aggregated endowments	factor payments (EVFP; v6 EVFA)
ESBT	intermediates/value added	intermediates, value added	intermediate purchases plus factor payments (v6: intermediate purchases, as FlexAgg's v6-format aggregation does)
ESBC	intermediate inputs	aggregated commodities	intermediate purchases
ESBG	government demand	aggregated commodities	government purchases
ESBI	investment demand	aggregated commodities	investment purchases
ETRQ	commodities made by an activity (CET)	aggregated commodities	make matrix (MAKS)

Header	Nest	Inputs (θ_k)	Value (V)
ESBQ	activities making a commodity	aggregated activities	make matrix (MAKB)
EFVE	value added/energy (GTAP-E, GTAP-Power)	value added, energy	factor payments plus energy purchases
EAEZ	land across agro-ecological zones (GTAP-AEZ)	aggregated zones	land rents

ELFVAEN for fossil fuel mining is recalibrated rather than averaged under either method (see below). The GTAP-E and GTAP-Power CDE parameters over `TOPP` are rebuilt from the commodity-level `SUBP`/`INCP`, the energy node as their private-consumption-weighted mean over the energy commodities, as flexagg's `aggpar_e.tab` does; the databases' own `SUBE`/`INCE` headers are not used.

6.6.2. GTAP v6.2 format parameter weight methodology

Headers, descriptions, and index ranges for parameters and associated data within the GTAP v6.2 model, with the `"value"` weights.

	Header	Description	Set Index
Param.			
σ_{d}	ESBD	Armington CES for dom./imp. allocation	$i \in$ TRAD_COMM
σ_{m}	ESBM	Armington CES for regional allocation of imports	$i \in$ TRAD_COMM
σ_{va}	ESBV	CES between primary factors in production	$j \in$ PROD_COMM
σ_{t}	ESBT	CES between intermediate inputs and value added	$j \in$ PROD_COMM
γ	INCP	CDE expansion parameter	$i \in$ TRAD_COMM, $r \in$ REG
β	SUBP	CDE substitution parameter	$i \in$ TRAD_COMM, $r \in$ REG
Data			
	EVFA	Endowments – Firms purchases at agents' prices	$i \in$ ENDW_COMM, $j \in$ PROD_COMM, $r \in$ REG
	VDFA	Inter. – Firms' dom. purchases at agents' prices	$i \in$ TRAD_COMM, $j \in$ PROD_COMM, $r \in$ REG
	VDGA	Inter. – Government dom. purchases at agents'	$i \in$ TRAD_COMM, $r \in$ REG

Header	Description	Set Index
VDPA	Inter. – Household dom. purchases at agents'	$i \in$ TRAD_COMM, $r \in$ REG
VIFA	Inter. – Firms' imp. at agents' prices	$i \in$ TRAD_COMM, $j \in$ PROD_COMM, $r \in$ REG
VIGA	Inter. – Government imp. at agents' prices	$i \in$ TRAD_COMM, $r \in$ REG
VIPA	Inter. – Household imp. at agents' prices	$i \in$ TRAD_COMM, $r \in$ REG

6.6.2.1. Aggregation methods

$$\hat{\sigma}_{\mathrm{d}_i} = \frac{\sum_i^{i^*} \left(\sigma_{\mathrm{d}_i} \sum_r (vdpa_{i,r} + vdga_{i,r} + vdfa_{i,r} + vipa_{i,r} + viga_{i,r} + vifa_{i,r})\right)}{\sum_{i,r}^{i^*} (vdpa_{i,r} + vdga_{i,r} + vdfa_{i,r} + vipa_{i,r} + viga_{i,r} + vifa_{i,r})} \tag{6.1}$$

$$\hat{\sigma}_{\mathrm{m}_i} = \frac{\sum_i^{i^*} \left(\sigma_{\mathrm{m}_i} \sum_r (vipa_{i,r} + viga_{i,r} + vifa_{i,r})\right)}{\sum_{i,r}^{i^*} (vipa_{i,r} + viga_{i,r} + vifa_{i,r})} \tag{6.2}$$

$$\hat{\sigma}_{\mathrm{va}_j} = \begin{cases} \dfrac{\sum_j^{j^*} \left(\sigma_{\mathrm{va}_j} \sum_r evfa_{j,r}\right)}{\sum_{j,r}^{j^*} evfa_{j,r}} & \text{if } j \neq cgds \\ \sigma_{\mathrm{va}_j} & \text{if } j = cgds \end{cases} \tag{6.3}$$

$$\hat{\sigma}_{\mathrm{t}_j} = \frac{\sum_j^{j^*} \left(\sigma_{\mathrm{t}_j} \sum_{i,r} (vdfa_{i,j,r} + vifa_{i,j,r})\right)}{\sum_{i,j,r}^{j^*} (vdfa_{i,j,r} + vifa_{i,j,r})} \tag{6.4}$$

$$\hat{\gamma}_{i,r} = \frac{\sum_{i,r}^{i^*r^*} \gamma_{i,r} \sum_{i,r} (vdpa_{i,r} + vipa_{i,r})}{\sum_{i,r}^{i^*r^*} (vdpa_{i,r} + vipa_{i,r})} \tag{6.5}$$

$$\hat{\beta}_{i,r} = \frac{\sum_{i,r}^{i^*r^*} \left(\beta_{i,r} \sum_{i,r} (vdpa_{i,r} + vipa_{i,r})\right)}{\sum_{i,r}^{i^*r^*} (vdpa_{i,r} + vipa_{i,r})} \tag{6.6}$$

6.6.3. GTAP v7 format parameter weight methodology

Headers, descriptions, and index ranges for parameters and associated data within the GTAP v7 model, with the `"value"` weights. The same weights apply to the other v7-format models, with GTAP-RE adding an invariant time set.

Header	Description	Set Index
Param.		

	Header	Description	Set Index
σ_{d}	ESBD	Armington CES for domestic/imported allocation	$c \in$ COMM
σ_{m}	ESBM	Armington CES for regional allocation of imports	$c \in$ COMM
σ_{va}	ESBV	CES between primary factors in production	$a \in$ ACTS, $r \in$ REG
σ_{t}	ESBT	CES between intermediate inputs and value added	$a \in$ ACTS, $r \in$ REG
σ_{c}	ESBC	CES between intermediate inputs	$a \in$ ACTS, $r \in$ REG
σ_{g}	ESBG	CES of government demand	$r \in$ REG
ω_{q}	ETRQ	CET between commodities produced by an activity	$a \in$ ACTS, $r \in$ REG
σ_{q}	ESBQ	CES between activities producing a commodity	$c \in$ COMM, $r \in$ REG
γ	INCP	CDE expansion parameter	$c \in$ COMM, $r \in$ REG
β	SUBP	CDE substitution parameter	$c \in$ COMM, $r \in$ REG
Data			
	EVFP	Primary factor purchases at purchasers' prices	$e \in$ ENDW, $a \in$ ACTS, $r \in$ REG
	VDFP	Domestic purchases by firms at purchasers' prices	$c \in$ COMM, $a \in$ ACTS, $r \in$ REG
	VDGP	Domestic purchases by government at purchasers' prices	$c \in$ COMM, $r \in$ REG
	VDPP	Domestic purchases by households at purchasers' prices	$c \in$ COMM, $r \in$ REG
	VMFP	Import purchases by firms at purchasers' prices	$c \in$ COMM, $a \in$ ACTS, $r \in$ REG
	VMGP	Import purchases by government at purchasers' prices	$c \in$ COMM, $r \in$ REG
	VMPP	Import purchases by households at purchasers' prices	$c \in$ COMM, $r \in$ REG
	VDIP	Domestic purchases by investment at purchasers' prices	$c \in$ COMM, $r \in$ REG
	VMIP	Import purchases by investment at purchasers' prices	$c \in$ COMM, $r \in$ REG
	MAKS	Make matrix at supply prices	$c \in$ COMM, $a \in$ ACTS, $r \in$ REG
	MAKB	Make matrix at basic prices	$c \in$ COMM, $a \in$ ACTS, $r \in$ REG

6.6.3.1. Aggregation methods

$$\hat{\sigma}_{\mathrm{d}_c} = \frac{\sum_c^{c^*} \left(\sigma_{\mathrm{d}_c} \sum_r (vdpp_{c,r} + vmpp_{c,r} + vdgp_{c,r} + vmgp_{c,r} + vdfp_{c,r} + vmfp_{c,r} + vdip_{c,r} + vmip_{c,r}) \right)}{\sum_{c,r}^{c^*} (vdpp_{c,r} + vmpp_{c,r} + vdgp_{c,r} + vmgp_{c,r} + vdfp_{c,r} + vmfp_{c,r} + vdip_{c,r} + vmip_{c,r})} \tag{6.7}$$

$$\hat{\sigma}_{\mathrm{m}_c} = \frac{\sum_c^{c^*} \left(\sigma_{\mathrm{m}_c} \sum_r (vmpp_{c,r} + vmgp_{c,r} + vmfp_{c,r} + vmip_{c,r}) \right)}{\sum_{c,r}^{c^*} (vmpp_{c,r} + vmgp_{c,r} + vmfp_{c,r} + vmip_{c,r})} \tag{6.8}$$

$$\hat{\sigma}_{\mathrm{va}_a} = \frac{\sum_a^{a^*} \left(\sigma_{\mathrm{va}_a} \sum_r evfp_{a,r} \right)}{\sum_{a,r}^{a^*} evfp_{a,r}} \tag{6.9}$$

$$\hat{\gamma}_{c,r} = \frac{\sum_{c,r}^{c^*r^*} \gamma_{c,r} \sum_{c,r} (vdpp_{c,r} + vmpp_{c,r})}{\sum_{c,r}^{c^*r^*} (vdpp_{c,r} + vmpp_{c,r})} \tag{6.10}$$

$$\hat{\beta}_{c,r} = \frac{\sum_{c,r}^{c^*r^*} \left(\beta_{c,r} \sum_{c,r} (vdpp_{c,r} + vmpp_{c,r}) \right)}{\sum_{c,r}^{c^*r^*} (vdpp_{c,r} + vmpp_{c,r})} \tag{6.11}$$

$$\hat{\sigma}_{\mathrm{t}_{a,r}} = \frac{\sum_{a,r}^{a^*r^*} \sigma_{\mathrm{t}_{a,r}} \left(\sum_c (vdfp_{c,a,r} + vmfp_{c,a,r}) + \sum_e evfp_{e,a,r} \right)}{\sum_{a,r}^{a^*r^*} \left(\sum_c (vdfp_{c,a,r} + vmfp_{c,a,r}) + \sum_e evfp_{e,a,r} \right)} \tag{6.12}$$

$$\hat{\sigma}_{\mathrm{c}_{a,r}} = \frac{\sum_{a,r}^{a^*r^*} \sigma_{\mathrm{c}_{a,r}} \sum_c (vdfp_{c,a,r} + vmfp_{c,a,r})}{\sum_{a,r}^{a^*r^*} \sum_c (vdfp_{c,a,r} + vmfp_{c,a,r})} \tag{6.13}$$

$$\hat{\sigma}_{\mathrm{g}_r} = \frac{\sum_r^{r^*} \sigma_{\mathrm{g}_r} \sum_c (vdgp_{c,r} + vmgp_{c,r})}{\sum_r^{r^*} \sum_c (vdgp_{c,r} + vmgp_{c,r})} \tag{6.14}$$

$$\hat{\omega}_{\mathrm{q}_{a,r}} = \frac{\sum_{a,r}^{a^*r^*} \omega_{\mathrm{q}_{a,r}} \sum_c maks_{c,a,r}}{\sum_{a,r}^{a^*r^*} \sum_c maks_{c,a,r}} \qquad \hat{\sigma}_{\mathrm{q}_{c,r}} = \frac{\sum_{c,r}^{c^*r^*} \sigma_{\mathrm{q}_{c,r}} \sum_a makb_{c,a,r}}{\sum_{c,r}^{c^*r^*} \sum_a makb_{c,a,r}} \tag{6.15}$$

6.6.4. GTAP-E, GTAP-Power and GTAP-AEZ parameters

The layer parameters are averaged, except for the following.

The elasticity of substitution between value added and energy, σ_{vae} (`EFVE`), is weighted by the share weight of the value-added/energy nest (`"share"`) or by the value of primary factors and energy inputs (`"value"`), where g runs over the energy commodities (`COME`):

$$\hat{\sigma}_{\text{vae}_{a,r}} = \frac{\sum_{a,r}^{a^*r^*} \sigma_{\text{vae}_{a,r}} vaen_{a,r}}{\sum_{a,r}^{a^*r^*} vaen_{a,r}}, \qquad vaen_{a,r} = \sum_{e} evfp_{e,a,r} + \sum_{g} (vdfp_{g,a,r} + vmfp_{g,a,r}) \tag{6.16}$$

For the fossil fuel mining activities (coal, oil and gas) it is instead recalibrated at the aggregate level so that the activity reproduces the target supply elasticity η (`SPLY`, averaged over all regions), given the cost shares of natural resources (s_{nr}) and of $vaen$ (s_{vaen}) in the value of output, assuming, as Burniaux and Truong (2002) do, a Leontief top nest and perfectly elastic supply of every input other than the natural resource:

$$\hat{\sigma}_{\text{vae}_{a,r}} = \frac{\eta_a}{1/s_{\text{nr}_{a,r}} - 1/s_{\text{vaen}_{a,r}}} \tag{6.17}$$

The other fuels (`p_c`, `gdt`) take $\hat{\sigma}_{\text{vae}} = 1$.

The elasticity of substitution between land in different agro-ecological zones, σ_{aez} (`EAEZ`), is weighted by land rents, the primary factor purchases $evfp_{e,a,r}$ over the zones $e \in$ `AEZS` (`"value"`), or by their share weight across zones (`"share"`).

7. Loading models

7.1. Overview

The `ems_model()` function loads the model and its closure, conducts pre-deployment checks, determines temporal dynamics, and allows for the loading of aggregated coefficient data. The output of this function is a tibble with attributes containing the parsed model input and is a required input to the `"model"` argument of the `ems_deploy()` function.

```
ems_model(
  model_file,
  closure_file,
  backsolve = NULL,
  ignore_condense = FALSE,
  ...
)
```

Argument	Type	Description
`model_file`	Character	File name in working directory or path to a `.tab` file
`closure_file`	Character	File name in working directory or path to a `.cls` closure file
`backsolve`	Character vector or `NULL`	Endogenous variables to substitute out by backsolving
`ignore_condense`	Logical	Ignore the `Substitute`, `Backsolve` and `Omit` statements found in the model file
`...`	Named arguments	Coefficient value injections (numeric, data frame, or CSV path)

Both `model_file` and `closure_file` are required arguments.

7.2. The model file

Model input in `teems` is limited to Tablo (`.tab`) files. Vetted models and their closures are available at teems-models, and all but ORANI-G also through `ems_example()`; their documentation is listed under References:

Model	Description	Database
GTAPv6	Classic GTAP model, version 6.2	GTAP v6.2 format
GTAPv7	Standard GTAP model, version 7.1	GTAP v7 format
GTAP-RE	Intertemporal GTAP with rational expectations: forward-looking investment, all periods solved at once	GTAP v7 format
GTAP-AEZ	GTAP with land use by agro-ecological zone: land enters production as a CES composite of the zones	GTAP-AEZ release
GTAP-E	GTAP with energy substitution and carbon taxes: a capital–energy composite inside value added	GTAP-E release
GTAP-EP	GTAP-E with electricity split into transmission and distribution and generation, and generation into base-load and peak-load technologies	GTAP-Power release
ORANI-G	Single-country model of the Centre of Policy Studies, 2013 edition	Its own `basedata.har`

Users may load their own models by providing a path to a Tablo file and closure:

```
model <- ems_model(
  model_file = "path/to/custom_model.tab",
  closure_file = "path/to/custom_model.cls"
)
```

When preparing a custom model, the vetted models are a good reference, and often the easiest starting point for modifications.

7.3. Closure selection

Standard model-specific closures are available within the model repository: teems-models. The general format consists of an `exogenous` block listing all (`\n` delimited) exogenous variables, followed by `;`, `rest endogenous`, and `;` to mark the end:

```
exogenous
pop
aoreg("row")
qe(ENDWC,REG,INITIME)
txs(MARG,"row",REG,FWDTIME)
...
;
rest endogenous
;
```

Each entry is either a full variable name (`pop`), an entry with specific set elements (`aoreg("row")`), a mixed entry (`txs(MARG,"chn",REG,FWDTIME)`) or an entry with subsets (`qe(ENDWC,REG,INITIME)`).

7.4. Tablo compatibility

The teems-solver follows GEMPACK's Tablo language, so most model files written for GEMPACK load as they are. This manual will not attempt to reproduce the very thorough GEMPACK documentation, rather lists what is supported and the few forms that are not. `ems_model()` checks the model file before anything is solved. Problems are reported by name, with the statement it occurs in, rather than surfacing later as a solver failure. If a model file does not load and you think it should, a supported statement behavior is not as expected, or an error propagates to the solver without warning, please get in touch.

7.4.1. Supported statements

`File`, `Coefficient`, `Read`, `Update`, `Set`, `Subset`, `Mapping`, `Formula`, `Assertion`, `Variable`, `Equation`, `Write`, `Zerodivide`, `Complementarity`, `Loop` with `Break` and `Cycle`, `PostSim` sections, and the condensation statements `Substitute` and `Backsolve` (see Condensation) are accepted. `Omit` statements hold the omitted variables exogenous and unshocked, as omission does in GEMPACK; they stay in the model and its outputs. Levels variables and equations, `Formula & Equation`, and `Read (IfHeaderExists)` are supported; levels equations may contain conditional sums, `PROD` and the functions `SQRT`, `EXP`, `LOGE`, `LOG10` and `IF`. `PROD`, `MAXS` and `MINS` reduce over a set like `SUM`. `Default` statements (`Variable (Default=Levels);`, `Equation (Default=Levels);`, `Formula (Default=Initial);`, `Coefficient (Default=Parameter);`) apply to the statements that follow them. `Read`, `Formula` and `Assertion` statements take effect in the order they appear, including inside `PostSim` sections: a `Read` after a default `Formula` replaces the default, and a `Formula` after a `Read` replaces what was read. A levels variable `X` has a linear variable named `p_X`, or `c_X` for a `(change)` variable; `Linear_Name=` gives it another name and `Linear_Var=` makes an already declared linear variable serve as it. In closures, shocks and swaps either name may be used. `p_X` and `c_X` refer to nothing else: beside a linear variable `X`, names such as `p_X` or `C_X` are free for other coefficients and variables.

`Display` and `Transfer` are not supported and raise an error: every coefficient is in the coefficient dump, and TEEMS writes no Header Array output files.

7.4.2. Names

Names are case-insensitive. A set, coefficient, variable, file or equation may be written in any case after its declaration, and TEEMS uses the declared spelling throughout, including in outputs. The same holds for header names (`Read ... header "Nat"` finds a header `NAT`), logical file names, the `.tab` and `.har` extensions, and the variable, set and element names in closures, shocks and swaps.

7.4.3. Variable types for homogeneity checks

`VPQType` marks a variable as a `Value`, `Price` or `Quantity`, or as `None` (tax rates, shifters, technology, foreign prices), for homogeneity tests (GEMPACK manual 57.2). All three GEMPACK forms are read, in file order:

```
Variable (begins p default VPQType Price);  ! later variables starting with p !
Variable (begins pf default VPQType None);  ! the longest matching prefix wins !
Variable (VPQType=Quantity) (all,i,IND) x1cap(i);
Variable (Name x6com VPQType Quantity);     ! anywhere in the file !
```

`Variable (begins <prefix> VPQType default OFF);` ends a default. Variables without a type stay `Unspecified`. `ORIG_LEVEL=` qualifiers are checked: the coefficient must be real and range over exactly the variable's sets.

7.4.4. Formulas, updates and mappings

- `Formula` quantifiers may carry general conditions, such as `(all,c,COM: $POS(c) = $POS(g))`, a coefficient on either side, or a mapping (`MAP(z) = r`). `Update` statements take quantifier conditions too; an element whose condition is false is left unchanged.
- A qualifier list such as `(initial, by_elements)` is accepted. `write updated value to file` is accepted with a warning but writes no separate file; the value is in the coefficient outputs.
- An index may be repeated within one reference (`T(d,d)`, `TRADE(c,"dom",r,r)`).
- A mapping may point into a subset of the set its argument position is declared over, and a mapped index may range over a subset of the mapping's domain. A mapping may also be assigned an identity or a copy (`MAP(c) = c`, `A(z) = B(z)`), matched by element name when the sets are unrelated.
- A mapping is read as element names (`Read (by_elements)`) or as element numbers (`Read` alone), over its whole domain or over a subset (`Read (all,c,MARG) MAP(c) ...`), with Formulas assigning the rest.
- `Update` statements read the values at the start of each step, and several updates of one element apply in order, the later overriding. `(change)` and `(explicit)` updated data follow the solution path and are extrapolated like the variables; see single-pass runs for models where that is not wanted.

7.4.5. Sets

Sets may be read from file, listed explicitly, or built from other sets:

```
Set A123 = A1 + A2 + A3;
Set B = (C - D) INTERSECT E;
Set B2 # equal to A # = A;
Set UACT = UNITA x ACTS;
Set COALCOMM # coal # =
    (all,c,COMM: sum{cc,DCOMM: MCOM(cc)=c, UNITDCOAL(cc)} > 0);
```

Set expressions (`UNION`, `INTERSECT`, `+`, `-`, and the relative complement `\`), set equality, set products and conditional set builders are supported. A few points to note:

- A set must not be named `union` or `intersect`.
- A conditional set builder, like `COALCOMM` above, compares an expression with a constant. Its source set may be read, listed or itself built from other sets. Sets are built before the model is solved, so the condition is evaluated at deployment from the loaded data: the coefficients in it must be read from a file or assigned by `Formula` statements that precede the `Set` statement and use only what is known by then (arithmetic, `IF`, sums, the standard functions); `RANDOM` cannot appear. GTAP-E's `UNITDCOAL(c) = 0.0 + IF[c="coa", 1.0]` is such a formula.
- Some GTAP models build sets from the `EFLG` and `SLUG` headers, which TEEMS excludes from the data. Declare such sets with explicit elements instead.
- Intertemporal set equality (`Set (Intertemporal) B = A;`) is not supported.
- A set may be empty, for example when a conditional set builder selects nothing under the data at hand. Statements over an empty set do nothing and sums over it are zero.

Set element names are lowercase throughout TEEMS (see Set mappings); for example, the v6.2 `CGDS` sector becomes `cgds`.

Element order follows the data file wherever a set is not aggregated (see Loading data). It matters to anything that works by position, such as `$POS` and a mapping read from a header, whose entries line up with the elements of its domain in order.

7.4.6. IF conditions

`IF[condition, expression]` is supported wherever an expression may stand, in formulas, updates, assertions, PostSim statements and linear or levels equations:

```
Formula (all,c,COMM)(all,r,REG)
    VCB(c,r) = VDB(c,r) + sum{d,REG, VXSB(c,r,d)} + IF[c in MARG, VST(c,r)];
Equation E_qca (all,c,COMM)(all,a,ACTS)(all,r,REG)
    qca(c,a,r) = IF[MAKES(c,a,r) gt 0,
                    qo(a,r) - ETRAQ(a,r) * [ps(c,a,r) - po(a,r)]];
Formula (all,r,REG) SHR(r) = sum{s,REG, IF[s <> r, VDB("food",s)]} / 2;
```

A condition may test set membership (`c in MARG`), compare an index with an element or another index (`d = "domestic"`, `r <> s`), or compare coefficient expressions (`MAKES(c,a,r) gt 0`). Comparisons may be joined with `AND`, `OR` and `NOT` (`[VDB(c,r) > 0] and not [r = "usa"]`), in `IF` terms as in the conditions of sums and quantifiers. `IF` terms may be nested, multiplied, divided, raised to powers, placed inside sums and function arguments, and used in formulas without quantifiers (`Formula D = 0.0 + IF[NM > 1, 1];`). Top-level terms of formulas and equations are rewritten at model load into equivalent unconditional statements, with helper coefficients and sets (named `IFC*`, `IFX*`, `IFT*`, `IFV*`, `IFO*`) that appear in outputs like any other coefficient or set; other placements are evaluated by the solver as written.

A condition reads coefficients and levels variables but not linear variables, and a set-membership test (`c in MARG`) cannot be combined with `AND`, `OR` or `NOT`; nest the `IF` instead.

7.5. Condensation

Condensation makes the system passed to the solver smaller without changing the model's economics. TEEMS condenses by backsolving variables.

The static vetted models come condensed. The GTAPv6, GTAPv7, GTAP-AEZ, GTAP-E and GTAP-EP model files carry the standard GTAP backsolves as `Backsolve` statements, and ORANI-G its own; `ems_model()` applies them on load. GTAP-RE carries none, because backsolving slows the parallel methods intertemporal models use (see below). To solve the full system, load a model with `ignore_condense = TRUE`.

7.5.1. Backsolving variables

A backsolved variable is substituted out of the system using its defining equation and recovered by the solver after each step, so it still appears in the outputs. By default the defining equation is `E_<variable>`; name it explicitly when it differs:

```
model <- ems_model(
  model_file = GTAPv7[["model_file"]],
  closure_file = GTAPv7[["closure_file"]],
  backsolve = c("pfd", "pfm", ps = "E_ps")
)
```

A backsolved variable must be endogenous, and its defining equation must be one GEMPACK could substitute with. If it is not, the error names the requirement that failed. #### Condensation coefficients (`CSUB`) {#csub}

Substituting a variable out multiplies the coefficients of its defining equation into every equation that used it. Rather than leave those products in the equation text, the engine stores each one, and each sum over an eliminated index, as a new coefficient with its own `Formula`, named `CSUB<n>` and appended to the deployed model file. Substituting `pint` out of GTAPv7 through `E_pint`, for example, produces

```
Coefficient (all,a,ACTS)(all,r,REG)(all,c,COMM)
    CSUB392(a,r,c) # backsolve product (E_pint) #;
Formula (all,a,ACTS)(all,r,REG)(all,c,COMM)
    CSUB392(a,r,c) = INTSHR(c,a,r)*[1 - FMSHR(c,a,r)];
```

and later substitutions build on earlier ones:

```
CSUB204(r,c) = sum{a,ACTS, CSUB53(r,c,a)*CSUB174(c,a,r)}
```

The comment names the kind: a *backsolve product* is such a product or sum; a *backsolve pivot* is the reciprocal of the coefficient on the backsolved variable when that coefficient is not a plain number.

They are derived from the model's coefficients, not declared by it, and are re-evaluated with the other formulas at every step. The standard GTAPv7 condensation adds 650 of them to the 283 coefficients the model declares, so the solver's write tally (`verbosity = 2`) reports the two groups separately. `ems_compose()` leaves them out of the default outputs and returns one only when it is named in `which`, and `write_coefficients = TRUE` writes no CSV for them.

7.5.2. When backsolving pays

Backsolving shrinks the system but makes it denser. That helps `"LU"` on static models: the standard GTAP condensation cut a 1.35 million equation Gragg solve by 41 % and a 3.6 million one by more than half. It hurts the parallel matrix methods (`"SBBD"`, `"DBBD"`, `"NDBBD"`), which rely on the sparsity it removes: on GTAP-RE the same condensation made `"SBBD"` runs 69 % slower at 200 thousand equations and 393 % slower at 4.4 million, and a condensed static system stops gaining from extra `"DBBD"` tasks where the full system keeps gaining. On a laptop, threaded `"LU"` on the condensed system is still the fastest measured below about 120 thousand condensed equations; above that, or on a machine with many cores, deploy without backsolve and use a parallel method. `ems_probe()` measures the block structure of a deployment and suggests which case it is likely in.

7.5.3. Condensation statements in the model file

`Substitute` and `Backsolve` statements in a model file are applied on load:

```
Substitute p1 using E_p1 ;
Backsolve p1lab using E_p1lab ;
```

`Substitute` is carried out as a backsolve, so the variable stays in the outputs. Statements in the file are applied first, then the `backsolve` argument. `ignore_condense = TRUE` skips the statements in the file but still applies the argument.

`Omit` statements add the listed variables to the closure as exogenous, unshocked entries, with a message. They stay in the model and its outputs, so a closure file need not repeat them.

Closure files may also carry `xSet` and `xSubset` lines as written in a GEMPACK command file; they are read as `Set` and `Subset` statements of the model, so closure entries can refer to them.

7.6. Coefficient value injection

The `...` argument in `ems_model()` allows users to assign aggregation-specific values to model coefficients. This is used to inject modified values into existing coefficients as well as provide data to new coefficients. All data inputted here must be model-aggregation specific and will not be subject to aggregation. The left-hand side (name) must correspond to a coefficient (not header) declared within the model input. Data frame and CSV inputs are validated against the set aggregations from `ems_data()`. Dimensions must match the aggregated resolution, not the original database resolution. Three input types are supported, with model “Read” and “Formula” declarations modified accordingly:

7.6.1. Uniform numeric value

A single numeric value can be assigned to a coefficient. All set combinations for that coefficient will receive this uniform value.

```
GTAP_RE <- ems_example("GTAP-RE", tempdir())
model <- ems_model(
  model_file = GTAP_RE[["model_file"]],
  closure_file = GTAP_RE[["closure_file"]],
  KAPPA = 0.54321
)
```

Providing a numeric vector of length greater than 1 will raise an error. Instead here use a data frame with the appropriate sets to assign multiple heterogeneous values to a coefficient.

7.6.2. Data frame

A data frame (or data frame extension such as tibble or data.table) can be used to assign heterogeneous values across set combinations. The data frame must contain columns for each set index using the model-specific naming convention (set name + variable index, e.g., `REGr`, `COMMc`, `ALLTIMEt`) and a `Value` column.

```
# Heterogeneous values for SUBPAR coefficient (Read-type)
GTAP_RE <- ems_example("GTAP-RE", tempdir())
COMMc <- c("crops", "food", "livestock", "mnfcs", "svces")
REGr <- c("usa", "chn", "row")
ALLTIMEt <- seq(0, 10)

SUBPAR <- expand.grid(
  COMMc = COMMc,
  REGr = REGr,
  ALLTIMEt = ALLTIMEt
)

SUBPAR$Value <- runif(nrow(SUBPAR))

model <- ems_model(
  model_file = GTAP_RE[["model_file"]],
  closure_file = GTAP_RE[["closure_file"]],
  SUBPAR = SUBPAR
)
```

This works for both `Read`-type and `Formula`-type coefficients:

```
# Heterogeneous values for CPHI coefficient (Formula-type)
REGr <- c("usa", "chn", "row")
ALLTIMEt <- seq(0, 10)

CPHI <- expand.grid(
  REGr = REGr,
```

```
  ALLTIMEt = ALLTIMEt
)
CPHI$Value <- runif(nrow(CPHI))

model <- ems_model(
  model_file = GTAP_RE[["model_file"]],
  closure_file = GTAP_RE[["closure_file"]],
  CPHI = CPHI
)
```

7.6.3. CSV file

A path to a CSV file can be used as an alternative to a data frame. The CSV must follow the same structure: set index columns and a `Value` column.

```
# Write coefficient data to CSV
write.csv(SUBPAR, "SUBPAR.csv", row.names = FALSE)

model <- ems_model(
  model_file = GTAP_RE[["model_file"]],
  closure_file = GTAP_RE[["closure_file"]],
  SUBPAR = "SUBPAR.csv"
)
```

7.6.4. Combining injections

Multiple coefficient injections can be combined in a single call:

```
model <- ems_model(
  model_file = GTAP_RE[["model_file"]],
  closure_file = GTAP_RE[["closure_file"]],
  KAPPA = 0.03,
  SUBPAR = SUBPAR,
  CPHI = CPHI
)
```

If the coefficient name is not found in the model, an error is raised indicating that the aggregated data coefficient is not declared within the provided model input.

7.7. Intertemporal models

For intertemporal models, `ems_model()` determines temporal dynamics by detecting intertemporal sets in the Tablo file. By default, the expected header in the GTAP-RE model is `"YEAR"`. If a model uses a different header for timestep data, set it via `ems_option_set()`:

```
ems_option_set(timestep_header = "AYRS")
```

If the expected timestep header is not found in the model, an error will be raised with a suggestion to use `ems_option_set()` to configure a custom timestep header. The same concept applies to the number of timesteps:

```
ems_option_set(n_timestep_header = "NINTERVAL")
```

Default values for these settings (and others) may be obtained using:

```
ems_option_get()
```

7.7.1. Leads, lags and parallel solving

In intertemporal models, the parallel matrix methods (see Solving the model) solve each period as a block, with the variables that link periods held in a shared border. Only the elements a lead or lag actually references join that border: `qo("capital",r,t+1)` adds the capital slice of `qo` and nothing else, and the solver's log reports the border it found (`Border netcut`). A lead or lag on a whole variable declared over large sets (for example `qo(c,r,t+1)` over all commodities and regions) does enlarge the border and can remove the benefit of solving in parallel.

Link periods through a small proxy variable in that case:

```
Variable (all,r,REG)(all,t,ALLTIME)
    kb(r,t) # beginning-of-period capital stock #;
Equation E_kb
(all,r,REG)(all,t,ALLTIME)
    kb(r,t) = qo("capital",r,t);
```

and place the lead on the proxy (`kb(r,t+1)`). The vetted intertemporal models already follow this pattern.

Part III.

Swaps and shocks

8. Closure swaps

8.1. Overview

Swaps allow the user to reconfigure which variables are exogenous (provided by the modeler) and which are endogenous (solved by the model). Simple full variable swaps can be directly inputted as a string into the `swap_in` and `swap_out` arguments of `ems_deploy()`. More complex swaps must be handled with the dedicated `ems_swap()` function.

8.2. Simple (full variable) swaps

Here, simple swaps refer to swaps in which a variable and all its components are being swapped. This is the most common type of swap and is directly handled within `ems_deploy()`.

A single full variable swap:

```
cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  swap_in = "qfd",
  swap_out = "tfd"
)
```

Multiple full variable swaps:

```
cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  swap_in = c("qfd", "yp"),
  swap_out = c("tfd", "dppriv")
)
```

Swaps "in" and "out" are reconciled together rather than in a fixed order: each variable element ends exogenous or endogenous according to the net of the swaps that name it. A variable can therefore be swapped in whole and a slice of it swapped back out, and a swap followed by its mirror leaves the closure unchanged. Swaps are rejected only when no order of them is valid on the closure, for example when the same element is swapped in twice.

8.2.1. Choosing swaps

A closure must leave as many endogenous variable elements as there are equations, so every element swapped in needs an element swapped out; `ems_deploy()` aborts when a closure is invalid. Each pair should also make economic sense: the variable swapped out takes up the adjustment that the newly fixed variable no longer can. In the examples above, exogenous `qfd` with endogenous `tfd` fixes a firm's demand for a domestic commodity and lets the tax on that purchase move to deliver it, the tax equivalent of a quantity target; a quota that may or may not bind needs a complementarity instead (Harrison et al. 2004). Exogenous `yp` with endogenous `dppriv` fixes private consumption expenditure and lets the distribution parameter of regional income adjust (Corong et al. 2017). Keep a price numeraire exogenous (`pfactwld` in the GTAP closures): the model determines relative prices only, and without one the price level is undetermined (Dixon et al. 1982).

8.3. Complex (partial variable) swaps

8.3.1. ems_swap()

`ems_swap()` prepares a partial variable swap for use in the `swap_in` or `swap_out` arguments of `ems_deploy()`. Set arguments in `...` use the model-specific set-index naming convention (set name concatenated with the variable's index letter, e.g. `REGr`, `ACTSa`) to identify which tuples to include in the swap.

```
ems_swap(
  var,
  ...
)
```

Argument	Type	Description
`var`	Character	Variable name as it appears in the model file
`...`	Named arguments	Set-index pairs restricting the swap. Each value is a single element (`"element"`), multiple elements (`c("a", "b")`), or a named subset (`"SUBSET"`)

Partial variable swaps entail swapping parts of a variable (i.e., subsets of the constitutive sets) and must be processed through the `ems_swap()` function. In the above example the full "qfd" and "yp" variables were made exogenous, while the full "tfd" and "dppriv" variables become endogenous. Within the GTAPv7 model, "qfd" encompasses the "COMM", "ACTS", and "REG" sets:

```
Variable (orig_level=VDFB)(all,c,COMM)(all,a,ACTS)(all,r,REG)
    qfd(c,a,r) # demand for domestic commodity c by activity a in region r #;
```

What if we wish to shock only certain parts of this variable (e.g., a region or commodity) and allow the remainder of the variable to endogenously adjust according to model equations? In this case we would swap on the parts of the variable that we will provide values to and leave the remainder. If, for example, we have a shock value for "asia" within "REG" and "crops" within "ACTS", the following `ems_swap()` call will prepare this swap for input into the `swap_in` argument of `ems_deploy()`:

```
qfd_in <- ems_swap(
  var = "qfd",
  REGr = "asia",
  ACTSa = "crops"
)
```

***Note that sets specified in this manner are identified by the model-specific concatenation of the standard set name plus the variable-specific index. This is necessary because many variables contain multiple instances of the same set and none of the `teems` functions relies upon input entry order or order within the model to make distinctions. In this case the "qfd" set subindices are "c", "a", and "r", yielding "COMMc", "ACTSa", and "REGr". In order to retain a valid closure we also use the same approach to the outgoing variable "tfd":

```
tfd_out <- ems_swap(
  var = "tfd",
  REGr = "asia",
  ACTSa = "crops"
)
```

While the subindices for both variables are the same within GTAPv7, note that these may vary in some models such as GTAPv6 where the REG subindex is "s" for "qfd" and "r" for "tfd". Both swaps may now be loaded within `ems_deploy()`:

```
cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  swap_in = qfd_in,
  swap_out = tfd_out
)
```

Or if additional full variable swaps are also desired, a list is used to load multiple swaps for each direction:

```
cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  swap_in = list(qfd_in, "yp"),
  swap_out = list(tfd_out, "dppriv")
)
```

Finally, selection of multiple elements for each set will expand the swap to encompass all associated tuples. For example:

```
qfd_in <- ems_swap(
  var = "qfd",
  REGr = c("asia", "lam"),
  ACTSa = "crops"
)
```

is equivalent to loading

```
qfd_in_1 <- ems_swap(
  var = "qfd",
  REGr = "asia",
  ACTSa = "crops"
)
```

and

```
qfd_in_2 <- ems_swap(
  var = "qfd",
  REGr = "lam",
  ACTSa = "crops"
)
```

8.3.2. Subset swaps

A named model subset may be passed in place of individual elements. `teems` expands the subset to all its elements at the aggregation level specified in `ems_data()`. This is useful when a model defines meaningful groupings that map directly to closure logic such as margin commodities or forward time steps in intertemporal models.

In the GTAP-RE model, `MARG` is a subset of `COMM` containing margin commodities, and `FWDTIME` is a subset of `ALLTIME` containing forward time steps. Passing these subset names directly restricts the swap to only those elements:

```
qxs_in <- ems_swap(
  var = "qxs",
  COMMc = "MARG",
  REGs = c("chn", "usa"),
  ALLTIMEt = "FWDTIME"
)

txs_out <- ems_swap(
  var = "txs",
```

```
  COMMc = "MARG",
  REGs = c("chn", "usa"),
  ALLTIMEt = "FWDTIME"
)

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = partial,
  swap_in = qxs_in,
  swap_out = txs_out
)
```

Note that `REGs = c("chn", "usa")` is a multi-element specification, while `COMMc = "MARG"` and `ALLTIMEt = "FWDTIME"` are subset specifications. Both the above forms can be combined freely in a single `ems_swap()` call.

9. Uniform shocks

9.1. Overview

`ems_uniform_shock()` applies a homogeneous shock over an entire variable or a subset of its elements. The set arguments accepted in `...` depend on the variable specified and the set mappings loaded in `ems_data()`.

```
ems_uniform_shock(
  var,
  value,
  ...
)
```

Argument	Type	Description
`var`	Character	Variable name as it appears in the model file
`value`	Numeric	Shock applied to all targeted elements: a percentage change for a percent-change variable, an ordinary change for a change variable
`...`	Named arguments	Set-index pairs restricting the shock. Each value is a single element (`"element"`), multiple elements (`c("a", "b")`), or a named subset (`"SUBSET"`)

9.2. Full variable uniform shocks

The simplest uniform shock is applied to a full variable, meaning that all combinations of sets associated with that variable are allocated the same exogenous value. In the example below, all regions (as determined by selected region mappings) will be increased by 1%.

```
pop_shk <- ems_uniform_shock(var = "pop",
                             value = 1
)
```

9.3. Partial variable uniform shocks

As with swaps, a uniform shock can be restricted to elements of the sets the variable is declared over, named by the set name concatenated with the variable's index (`REGr`, `ACTSa`). To shock the region "chn" alone:

```
partial <- ems_uniform_shock(var = "aoall",
                             value = -1,
                             REGr = "chn"
)
```

Exogenous elements the shock does not name (here every region but "chn") keep a zero change.

In an intertemporal model the time element may be given as a chronological year, `Year = 2017`, or as a time step through the time set (see the "Time step summary" that `ems_data()` prints):

```
partial <- ems_uniform_shock(
  var = "aoall",
  value = -1,
  REGr = "chn",
  ACTSa = "crops",
  Year = 2017
)
```

9.4. Subset shocks

A named model subset may be passed in place of individual elements, identical to the subset form in `ems_swap()`. `teems` expands the subset to all its elements at the aggregation level specified in `ems_data()`.

In the GTAP-RE model, `MARG` is a subset of `COMM` containing margin commodities, and `FWDTIME` is a subset of `ALLTIME` containing forward time steps. Using these subset names directly restricts the shock to only those elements:

```
partial <- ems_uniform_shock(
  var = "qxs",
  value = -1,
  COMMc = "MARG",
  REGs = c("chn", "usa"),
  ALLTIMEt = "FWDTIME"
)
```

As in the swaps example, `REGs = c("chn", "usa")` is a multi-element specification while `COMMc = "MARG"` and `ALLTIMEt = "FWDTIME"` are subset specifications. This shock would then be deployed alongside the corresponding subset swaps:

```
cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = partial,
  swap_in = qxs_in,
  swap_out = txs_out
)
```

9.5. Multiple shocks and mixed swaps

Multiple uniform shocks of different scope can be combined in a list and passed to `ems_deploy()`. Swaps can also be mixed, combining partial `ems_swap()` objects with full variable strings (GTAPv6 sets in this example):

```
partial <- ems_uniform_shock(
  var = "qfd",
  value = -1,
  REGs = "usa",
  PROD_COMMj = "crops"
)

full <- ems_uniform_shock(
  var = "yp",
  value = 0.1
)

qfd <- ems_swap(
  var = "qfd",
  REGs = "usa",
  PROD_COMMj = "crops"
)

tfd <- ems_swap(
  var = "tfd",
  REGr = "usa",
  PROD_COMMj = "crops"
)

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = list(partial, full),
  swap_in = list(qfd, "yp"),
  swap_out = list(tfd, "dppriv")
)
```

10. Custom shocks

10.1. Overview

`ems_custom_shock()` allows for heterogeneous shocks specified on a tuple-by-tuple basis, supplied as a data frame or CSV file through `input`. Values are percentage changes for percent-change variables and ordinary changes for change variables. Only the tuples present in `input` are shocked.

```
ems_custom_shock(
  var,
  input
)
```

Argument	Type	Description
`var`	Character	Variable name as it appears in the model file
`input`	Data frame or Character	Data frame or path to a CSV file containing set columns and a `Value` column (the shock in the variable's own form)

10.2. Full variable custom shocks

Custom shocks can be carried out over the full variable, with every tuple receiving a specified shock value. Here is a demonstration of 4 custom shocks carried out on all variable tuples for a range of variable dimensions with the GTAP-RE model.

Elements for data frame construction

```
time_steps <- c(0, 1, 2, 3)
REG <- c("chn", "usa", "row")
ENDW <- c("labor", "capital", "natlres", "land")
COMM <- c("svces", "food", "crops", "mnfcs", "livestock")
ACTS <- COMM
MARG <- "svces"
ALLTIME <- seq(0, length(time_steps) - 1)
```

2D data frame and shock

```
pop <- expand.grid(
  REGr = REG,
  ALLTIMEt = ALLTIME,
  stringsAsFactors = FALSE
)

pop <- pop[do.call(order, pop), ]
pop$Value <- runif(nrow(pop))

pop_shk <- ems_custom_shock(
  var = "pop",
  input = pop
)
```

3D data frame and shock

```
aoall <- expand.grid(
  ACTSa = ACTS,
  REGr = REG,
  ALLTIMEt = ALLTIME,
  stringsAsFactors = FALSE
)

aoall <- aoall[do.call(order, aoall), ]
aoall$Value <- runif(nrow(aoall))

aoall_shk <- ems_custom_shock(
  var = "aoall",
  input = aoall
)
```

4D data frame and shock

```
afeall <- expand.grid(
  ENDWe = ENDW,
  ACTSa = ACTS,
  REGr = REG,
  ALLTIMEt = ALLTIME,
  stringsAsFactors = FALSE
)

afeall <- afeall[do.call(order, afeall), ]
afeall$Value <- runif(nrow(afeall))

afeall_shk <- ems_custom_shock(
  var = "afeall",
```

```
  input = afeall
)
```

5D data frame and shock

```
atall <- expand.grid(
  MARGm = MARG,
  COMMc = COMM,
  REGs = REG,
  REGd = REG,
  ALLTIMEt = ALLTIME,
  stringsAsFactors = FALSE
)

atall <- atall[do.call(order, atall), ]
atall$Value <- runif(nrow(atall))

atall_shk <- ems_custom_shock(
  var = "atall",
  input = atall
)
```

Multiple custom shocks are combined in a list for `ems_deploy()`:

```
cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = list(pop_shk, aoall_shk, afeall_shk, atall_shk)
)
```

10.3. CSV input

A path to a CSV file can be passed to `input` instead of a data frame. The CSV must follow the same structure: one column per set index using the model-specific naming convention and a `Value` column. Column order is inconsequential.

```
pop_csv <- tempfile(fileext = ".csv")
write.csv(pop, pop_csv, row.names = FALSE)

pop_shk <- ems_custom_shock(
  var = "pop",
  input = pop_csv
)
```

10.4. Partial variable custom shocks

Custom shocks are only carried out on tuples included within the input data frame or CSV. Here we carry out a custom shock (heterogenous values) across part of the `aoall` variable. The non-shocked values will remain at `0` due to their exogenous status.

```
aoall_full <- expand.grid(
  ACTSa = ACTS,
  REGr = REG,
  ALLTIMEt = ALLTIME,
  stringsAsFactors = FALSE
)
aoall_full$Value <- 0
aoall_full <- aoall_full[do.call(order, aoall_full), ]
aoall <- aoall_full[
  !(aoall_full$ACTSa == "crops" & aoall_full$REGr == "usa") &
  aoall_full$ALLTIMEt != 3,
]
aoall$Value <- runif(nrow(aoall))
aoall_shk  <- ems_custom_shock(var = "aoall",  input = aoall)
```

10.5. Chronological year input

For intertemporal models, the time dimension can be specified using chronological years instead of integer time steps by using a `Year` column in place of `ALLTIMEt`. The year values must correspond to the reference year and steps defined in `ems_data()`.

```
time_steps <- c(2017, 2018, 2019, 2020)

dat <- ems_data(
  dat_input = dat_input,
  par_input = par_input,
  set_input = set_input,
  time_steps = time_steps,
  REG = "big3",
  ACTS = "macro_sector",
  ENDW = "labor_agg"
)
```

Construct the input data frame with a `Year` column rather than `ALLTIMEt`:

```
REG <- c("chn", "usa", "row")

pop <- expand.grid(
  REGr = REG,
```

```
  Year = time_steps,
  stringsAsFactors = FALSE
)

pop <- pop[do.call(order, pop), ]
pop$Value <- runif(nrow(pop))

pop_shk <- ems_custom_shock(
  var = "pop",
  input = pop
)
```

With `ems_custom_shock()`, a `Year` column is recognized automatically and mapped to the corresponding integer time step internally.

11. Scenario shocks

11.1. Overview

`ems_scenario_shock()` is an intertemporal shock type that specifies heterogeneous shocks at the tuple level as absolute values. By default, inputs must cover all pre-aggregation tuples and span the full extent of the variable; with `partial = TRUE` they may cover only some of the aggregated tuples (see Partial scenario shocks). Values must be in the same units as the underlying database coefficient; at deployment they are summed over the set mappings of `ems_data()`, and each year's aggregated value becomes the percentage change from the base-year value (the ordinary change for a variable declared `(change)`), so the same input serves any aggregation.

```
ems_scenario_shock(
  var,
  input,
  partial = FALSE
)
```

Argument	Type	Description
`var`	Character	Variable name as it appears in the model file
`input`	Data frame or Character	Data frame or path to a CSV file containing all pre-aggregation tuples, a `Year` column (chronological years), and a `Value` column (absolute values)
`partial`	Logical	If `TRUE`, `input` may cover only some aggregated tuples; only those are shocked (default `FALSE`)

11.2. Using scenario shocks

Scenario shocks differ from custom shocks in a number of respects:

1. Only available for intertemporal models
2. Inputs must contain all *preaggregation* tuples associated with a variable, unless `partial = TRUE`
3. Inputs are denominated in absolute values in the units of the underlying database coefficient (e.g., million $USD, population)

4. The input data frame or CSV must contain a `Year` column of chronological years, the base year included, since each later year's shock is the change from the base-year value

There are several applications where it is advantageous to use scenario shock rather than a custom shock. The primary advantage is that scenario shocks allow for seamless changes to model aggregations since the shocks themselves are subject to mappings and aggregation. If we have trajectories available for all tuples within a given variable, a scenario shock will adjust to set mapping inputs in `ems_data()`. Because values are summed over the pre-aggregation tuples before the percentage change is taken, they must be levels in common real units (e.g., persons, million USD). Index values such as a base of 1 for every country would give an aggregated region the unweighted mean of its members' growth rather than their size-weighted growth; they are safe only when no tuple is aggregated.

11.3. Population trajectory example

Here is an example that takes base year population data from the GTAP database (typically not read into the model) and constructs hypothetical pathways for every region in the model. These pathways are valid for any regional aggregation chosen.

Set up the model run with chronological time steps:

```
year <- 2017
time_steps <- year + c(0, 1, 2, 3)

dat <- ems_data(
  dat_input = "v7_data/gsdfdat.har",
  par_input = "v7_data/gsdfpar.har",
  set_input = "v7_data/gsdfset.har",
  time_steps = time_steps,
  REG = "big3",
  ACTS = "macro_sector",
  ENDW = "labor_agg"
)

GTAP_RE <- ems_example("GTAP-RE", tempdir())
model <- ems_model(
  model_file = GTAP_RE[["model_file"]],
  closure_file = GTAP_RE[["closure_file"]]
)
```

Load POP at full regional resolution (unaggregated) as the scenario baseline. The `REG = "full"` call does not require `time_steps` since it is only used to extract the base-year coefficient:

```
pop <- ems_data(
  dat_input = "v7_data/gsdfdat.har",
  par_input = "v7_data/gsdfpar.har",
```

```
  set_input = "v7_data/gsdfset.har",
  REG = "full",
  ACTS = "macro_sector",
  ENDW = "labor_agg"
)$POP
```

Construct trajectories with compound growth through each future time step. `tail(time_steps, -1)` yields the forward time steps, keeping the trajectory in sync with whatever `time_steps` was passed to `ems_data()`:

```
pop$Year <- year
regions <- unique(pop$REG)

pop_traj <- expand.grid(REG = regions, Year = tail(time_steps, -1),
                        stringsAsFactors = FALSE)
pop_traj$Value <- 0
pop <- rbind(pop, pop_traj)

set.seed(42)
growth_rates <- data.frame(
  REG = regions,
  growth_rate = runif(length(regions), min = -0.01, max = 0.05)
)

pop <- merge(pop, growth_rates, by = "REG")
base_values <- pop[pop$Year == year, c("REG", "Value")]
names(base_values)[2] <- "base_value"
pop <- merge(pop, base_values, by = "REG")

pop$Value[pop$Year > year] <-
  pop$base_value[pop$Year > year] *
    (1 + pop$growth_rate[pop$Year > year])^(pop$Year[pop$Year > year] - year)

pop$growth_rate <- NULL
pop$base_value <- NULL
pop <- pop[order(pop$REG, pop$Year), c("REG", "Year", "Value")]
```

`ems_data()` returns coefficients with plain set names (e.g., `REG`). Rename to the model's set-index convention before passing to `ems_scenario_shock()`:

```
colnames(pop)[colnames(pop) == "REG"] <- "REGr"
```

Pass `pop` as the scenario shock and deploy:

```
pop_trajectory <- ems_scenario_shock(
  var = "pop",
  input = pop
)

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = pop_trajectory
)
```

`pop` may also be saved as a CSV and loaded directly by passing the file path to `input`.

11.4. Partial scenario shocks

Sometimes trajectories exist for only part of a variable, for example the population of a single region. With `partial = TRUE`, a scenario shock covers only the aggregated tuples its input fully supplies. Every pre-aggregation tuple that maps to a shocked aggregated tuple must be present, in every year. An aggregated tuple that is only partly covered stops the deployment by name, because the percentage change of a partial sum is not the change of the whole group.

Continuing the example above, the following shocks population in the United States and China only, which are their own regions in the `"big3"` aggregation:

```
pop_partial <- pop[pop$REGr %in% c("usa", "chn"), ]

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = ems_scenario_shock(var = "pop", input = pop_partial, partial = TRUE)
)
```

The shocked tuples must be exogenous, and exogenous tuples outside the input keep a zero change. The same input stops under an aggregation that groups the United States with other countries, unless those countries are supplied as well. Since values are summed over the pre-aggregation tuples, scenario shocks, partial or not, suit additive variables such as values and quantities.

Part IV.

Deploying and solving the model

12. Deploying the model

12.1. Overview

`ems_deploy()` validates all data and model inputs, processes shocks and closure swaps, and writes all required input files to disk. The output file path serves as a required input to `ems_solve()`.

```
ems_deploy(
  .data,
  model,
  shock = NULL,
  swap_in = NULL,
  swap_out = NULL,
  shock_file = NULL,
  write_coefficients = FALSE
)
```

12.2. Arguments

Argument	Default	Description
`.data`		A list of data.tables generated by `ems_data()`
`model`		A tibble generated by `ems_model()`
`shock`	`NULL`	A shock object or list of shock objects produced by `ems_uniform_shock()`, `ems_custom_shock()`, or `ems_scenario_shock()`. When `NULL`, no shock is applied and input data is returned without change
`swap_in`	`NULL`	A character vector, list generated by `ems_swap()`, or combination provided as a list. Variables to be made exogenous
`swap_out`	`NULL`	A character vector, list generated by `ems_swap()`, or combination provided as a list. Variables to be made endogenous

Argument	Default	Description
`shock_file`	`NULL`	Path to a file with `.shf` extension representing a fully prepared shock file. No checks or modifications are carried out on this file
`write_coefficients`	`FALSE`	Coefficient values return to R through the solver's binary coefficient dump, read selectively by `ems_compose()`. Set `TRUE` to also emit per-coefficient CSV files (`out/coefficients/`) via auto-generated `File/Write` statements, e.g. for inspection outside R. PostSim coefficients are written there too, after the PostSim pass; the condensation coefficients (`CSUB<n>`) are dump-only

12.3. Basic usage

The simplest deployment requires only data and model inputs. With no shock supplied, input data is returned without change:

```
cmf_path <- ems_deploy(
  .data = dat,
  model = model
)
```

12.4. Deploying with shocks and swaps

A single shock can be passed directly:

```
shock <- ems_uniform_shock(var = "pop", value = 1)

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = shock
)
```

Multiple shocks are combined in a list:

```
pop_shk <- ems_uniform_shock(var = "pop", value = 1)
aoall_shk <- ems_custom_shock(var = "aoall", input = aoall_df)

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = list(pop_shk, aoall_shk)
)
```

12.5. Closure swaps

A single full variable swap swaps one variable in and one out using plain strings:

```
shock <- ems_uniform_shock(var = "qfd", value = 1)

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = shock,
  swap_in = "qfd",
  swap_out = "tfd"
)
```

Multiple full variable swaps are passed as character vectors. Partial swaps use `ems_swap()` objects and can be mixed with strings in a list:

```
# Multiple full variable swaps
cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = shock,
  swap_in = c("qfd", "yp"),
  swap_out = c("tfd", "dppriv")
)

# Mixed partial and full swaps
yp_row <- ems_swap("yp", REGr = "row")
dppriv_row <- ems_swap("dppriv", REGr = "row")

cmf_path <- ems_deploy(
  .data = dat,
  model = model,
  shock = shock,
  swap_in = list(yp_row, "qfd"),
```

```
  swap_out = list(dppriv_row, "tfd")
)
```

12.6. User-generated inputs

`ems_deploy()` allows for user-generated shock files via the `shock_file` argument. In the case of shock files, no checks or operations will be conducted (i.e., the shock file as loaded is considered the final input). User-supplied closure files will however be processed if swaps are supplied.

13. Solving the model

13.1. Overview

`ems_solve()` runs the teems solver on a deployed model and, by default, parses the results into structured R objects with `ems_compose()`. A teems Docker image must be available. After the run, the solve is checked for singularity, accuracy and errors, and the findings are written to `model_diagnostics.txt` in the run directory. On Linux the solver container runs as the calling user, so the files it writes to the run directory belong to you.

`ems_solve()` chooses nothing on your behalf: the run is exactly the arguments given, so it can be reproduced from its record. A bare `ems_solve(cmf_path)` solves with Gragg 2-4-6 (as GEMPACK) and LU on one task and one thread, which is accurate and correct for every model but slow for large ones. `ems_probe()` suggests what to use instead.

```
ems_solve(
  cmf_path,
  solution_method = c("Gragg", "Johansen", "Euler", "Midpoint", "RK2",
                      "Heun", "RK4", "BoSha32", "DoPri54"),
  matrix_method = c("LU", "DBBD", "SBBD", "NDBBD"),
  n_subintervals = 1L,
  steps = NULL,
  n_tasks = 1L,
  n_threads = 1L,
  precision = c("single", "double"),
  verbosity = 1L,
  suppress_outputs = FALSE,
  terminal_run = FALSE,
  accuracy_vars = NULL,
  ...
)
```

13.2. Basic usage

```
outputs <- ems_solve(
  cmf_path = cmf_path,
  solution_method = "Gragg",
  matrix_method = "LU"
)
```

13.3. Arguments

Argument	Default	Description
`cmf_path`		Path to the CMF file generated by `ems_deploy()`
`solution_method`	`"Gragg"`	How the shock is applied (see Solution methods)
`matrix_method`	`"LU"`	How each linear system is solved (see Matrix methods)
`n_subintervals`	`1L`	Splits the shock into this many parts, each solved in turn. Helps with large shocks
`steps`	`c(2L, 4L, 6L)`	Step counts. For `"Gragg"`, `"Midpoint"` and `"Euler"`, two or three strictly increasing counts are extrapolated, and a single count is one multi-step run without extrapolation (see Single-pass runs); a single count for the Runge-Kutta methods
`n_tasks`	`1L`	Parallel tasks. Must be `1L` with `"LU"`
`n_threads`	`1L`	Threads per task
`precision`	`"single"`	`"double"` uses the double-precision solver: about twice the memory, for runs with many steps or a near-singular system
`verbosity`	`1L`	Log detail: `0` errors, warnings and the accuracy summary; `1` adds progress and timings; `2` adds per-rank detail and a one-line tally of what was written (coefficients, sets, variables)
`suppress_outputs`	`FALSE`	When `TRUE`, outputs are not parsed; the CMF path is returned invisibly for a later `ems_compose()`
`terminal_run`	`FALSE`	When `TRUE`, prepares the run without starting it, so it can be launched from a terminal

`...` \| \| Named extras: the Runge-Kutta controls (`adaptive`, `eps_tolerance`, `max_retries`, `retry_adjust`; see Runge-Kutta controls), the MA48 workspace guesses (`laA`, `laD`, `laDi`) and expert solver flags (e.g. `condest`, `inmemory`, `postsim`). Unknown names are an error \|

The severities of TAB `Assertion` failures and declared-range violations are package options (`assertions`, `range_test_initial`, `range_test_updated`).

13.4. Solution methods

Method	Use it for
`"Gragg"`	The default and the standard accurate method: several multi-step solutions combined by extrapolation (see the GEMPACK manual)
`"Johansen"`	A quick one-step linear approximation. Good for checking that a model and closure run; not accurate for nonlinear results or large shocks
`"Euler"`	Like `"Gragg"` but converges more slowly. With adaptive `"DoPri54"` or `"BoSha32"`, the choice for severe shocks, such as a variable driven close to `-100%`, where `"Gragg"` and `"Midpoint"` can become unstable (GEMPACK manual 30.2; Schiffmann 2022)
`"Midpoint"`	Gragg's method without its final smoothing pass. It never carries the shocks past their end point, so it takes a shock of exactly `-100%`, which `"Gragg"` refuses
`"RK2"`, `"Heun"`, `"RK4"`	Runge-Kutta methods with a single step count and no extrapolation. `"RK4"` with a few steps is an inexpensive way to get three to four correct digits on large shocks
`"BoSha32"`, `"DoPri54"`	Runge-Kutta methods that estimate their own error and can control their step size (`adaptive = "yes"`)

`"Gragg"` suits most work; `"Johansen"` is a quicker way to check that a new setup runs, and adaptive `"DoPri54"` is an alternative for large shocks. A shock that is too large for the step schedule shows up as a singularity abort preceded by warnings that updated coefficients crossed their declared bounds; `ems_solve()` then names the remedies below rather than the closure. On large shocks, use more than one subinterval with `"Gragg"`:

```
outputs <- ems_solve(
  cmf_path = cmf_path,
  solution_method = "Gragg",
  n_subintervals = 4,
  steps = c(2, 4, 8)
)
```

With `matrix_method = "DBBD"`, a run split into subintervals can occasionally abort with a singularity partway through the shock although the closure is sound; the solver names the elements involved. Change `n_subintervals` (`1` is the simplest) or use `matrix_method = "LU"` for that run.

Adaptive `"DoPri54"` reaches about the same accuracy with roughly half the work. It controls its own step size, so `n_subintervals` does not apply, and it returns a per-element `error_estimate` column. Treat the estimate as a pointer to the least settled results, not as a bound on the error.

```
outputs <- ems_solve(
  cmf_path = cmf_path,
  solution_method = "DoPri54",
  steps = 4,
  adaptive = "yes",
  eps_tolerance = 0.01
)
outputs$dat$qgdp$error_estimate
```

To check that a result has converged, repeat the run with a tighter `eps_tolerance` or more steps and compare. The finer Runge-Kutta controls are described below.

13.4.1. Step counts and single-pass runs

`"Gragg"`, `"Midpoint"` and `"Euler"` normally solve three times with increasing step counts and extrapolate from the three results, which also gives the accuracy summary. The summary covers every endogenous result unless `accuracy_vars` names the variables it should cover, for example `accuracy_vars = c("qgdp", "pm")`. The `"Gragg"` and `"Midpoint"` counts must be all even or all odd (`c(2, 4, 6)` or `c(3, 5, 7)`); even counts are recommended. Two counts (`c(2, 4)`) extrapolate from two results, with no accuracy summary. The package option `converge = TRUE` applies GEMPACK's treatment of results that do not converge cleanly: their extrapolation is replaced by the result with the most steps, or by the average when the three results are nearly equal.

A single count gives one multi-step run without extrapolation:

```
outputs <- ems_solve(
  cmf_path = cmf_path,
  solution_method = "Euler",
  steps = 16
)
```

Use it for models whose updated data must follow the steps themselves, such as a step counter or an iterative (RAS-style) balancing update, which extrapolation would distort. No accuracy summary is available for a single-pass run.

13.4.2. Runge-Kutta controls

```
ems_RK(
  cmf_path,
  solution_method = c("DoPri54", "BoSha32", "RK4", "Heun", "RK2"),
  steps = 4L,
  eps_tolerance = 0.01,
  adaptive = NULL,
  matrix_method = c("LU", "DBBD", "SBBD", "NDBBD"),
```

```
  n_tasks = 1L,
  n_threads = 1L,
  precision = c("single", "double"),
  n_subintervals = 1L,
  verbosity = 1L,
  suppress_outputs = FALSE,
  terminal_run = FALSE,
  accuracy_vars = NULL,
  max_retries = 3L,
  retry_adjust = 0.5,
  chart = c("log", "percent"),
  error_norm = c("max", "rms"),
  controller = c("std", "pi"),
  scope = c("pct", "all"),
  h_init = NULL,
  guard = NULL,
  ...
)
```

`ems_RK()` is `ems_solve()` for the Runge-Kutta methods. It defaults to adaptive `"DoPri54"` and adds the step-size controls; the remaining arguments are those of `ems_solve()`, with the same defaults. When most retries come from step checks (a variable approaching `-100%`, a range test) rather than from `eps_tolerance`, the cheaper steps of `"BoSha32"` can make it faster than `"DoPri54"` (Schiffmann 2022).

```
# adaptive DoPri54
outputs <- ems_RK(cmf_path)

# fixed-step RK4
outputs <- ems_RK(cmf_path, solution_method = "RK4", steps = 8L)
```

Argument	Default	Description
`solution_method`	`"DoPri54"`	`"DoPri54"`, `"BoSha32"`, `"RK4"`, `"Heun"` or `"RK2"`
`steps`	`4L`	Number of steps; under adaptive control, the starting number
`adaptive`	`"yes"` for `"DoPri54"`/`"BoSha32"`, otherwise `"no"`	`"yes"` retries steps that miss `eps_tolerance` or fail a step check (a variable crossing `-100%`, a range test or an assertion, as in GEMPACK) and adjusts the step size; `"accuracy-only"` ignores the step checks

Argument	Default	Description
`eps_tolerance`	`0.01`	Accuracy target per step. GEMPACK suggests `0.1` for most simulations; on large shocks the default `0.01` reaches four correct digits on most elements, and `0.001` gains the last digits at about 1.6 times the cost
`max_retries`, `retry_adjust`	`3L`, `0.5`	How often, and by how much, a step is shortened when it fails a step check
`chart`	`"log"`	`"log"` keeps every level positive; `"percent"` combines stages in percentage changes
`error_norm`	`"max"`	Whether the worst element (`"max"`) or the root mean square (`"rms"`) decides if a step is accepted
`controller`	`"std"`	`"pi"` rejects fewer steps on rough solution paths
`scope`	`"pct"`	Which variables steer the step size: percentage-change variables (`"pct"`) or all variables (`"all"`)
`h_init`, `guard`	`NULL`	First step length, at most `1/steps` (a longer one is refused); and the level ratio at which a stage is rejected under `chart = "log"`

The defaults suit most runs.

13.5. Matrix methods

Method	Models	Description
`"LU"`	all	Sequential LU factorization. The most robust; one task only
`"DBBD"`	static	Splits the system into blocks (usually by region) solved in parallel, treating regions as separable economies (Kompas and Van Ha 2019)
`"SBBD"`	intertemporal	Splits the system by time period, solved in parallel (Van Ha and Kompas 2016; Van Ha et al. 2017)
`"NDBBD"`	intertemporal	A nested split for intertemporal models too large for `"SBBD"` in the available memory

13.5.1. Choosing a method and resources

```
ems_probe(
  cmf_path,
  components = TRUE,
  cores = NULL,
  memory = NULL,
  ...
)
```

Run `ems_probe()` once per deployment, before the first solve. It runs the solver up to the point where the equation system is assembled, without solving, and from that system it checks that the closure is sound, measures the structure the matrix methods exploit, reads the cores and memory available to the solver's container, and prints the call to use. A probe takes seconds on a condensed GTAP model; on an uncondensed system its cost grows faster than the size, to minutes at a few million equations.

```
probe <- ems_probe(cmf_path)
#>   Probing 'GTAPv7.cmf'; solver log at 'out/probe/solver_out_probe.txt'
#>   Probe finished in 20.9 s
probe
#>   Structural probe                      110,849 equations
#>   Closure valid: every variable is determined and no equation is redundant,
#>   structurally and at base data
#> → Recommended for this container (8 cores, 12.0 GB): "LU", 1 task × 8
#>   threads
#>   `ems_solve(cmf_path, matrix_method = "LU", n_threads = 8)`
#>   `summary(x)` for the full report
```

Pass `cores` and `memory` (in GB) to get the recommendation for another machine, for example a server you will solve on later.

`ems_probe()` is experimental. Its rules come from a limited set of measured models and machines, so treat the recommendation as a starting point: a different method, task count or thread count may well be faster for your model and machine. Try other settings with `ems_solve()`, and if a suggestion is off, please open an issue.

13.5.1.1. Reading the printed summary

While it runs, the two progress lines name the deployment, the solver log and the time taken; any `Warning:` lines the solver wrote to that log are repeated afterwards. Both are silent under `ems_option_set(verbose = FALSE)`. The short form printed above has four parts:

- **The header** gives the number of equations in the system the solver will factorize. For a condensed model (the vetted static models) this is the system after backsolving, which is much smaller than the model.

- **The closure verdict.** `Closure valid` means that every endogenous variable is pinned down by some equation and no equation duplicates another, both in the model's structure and with the base data in place. A model that fails this test cannot be solved, however it is configured; the alternative lines are described under Singular systems.
- **A memory warning**, only when the estimated peak of the recommended run comes close to the container's memory.
- **The recommendation**: the matrix method, the number of parallel tasks and the threads per task, for the machine named in brackets, followed by the `ems_solve()` call to paste. "This container" means the cores and memory were read from the Docker container the solver runs in (on Docker Desktop, the limits under Settings > Resources); "the machine given" means you passed `cores` and `memory`.

When no sound recommendation can be made, the probe says so instead of guessing: the closure is singular (fix the model first), the container's cores could not be read (pass `cores` and `memory`), or no method fits in the memory available (the message gives the smallest estimate and the remedies: more memory for the container, a coarser aggregation, or condensation).

13.5.1.2. The full report

`summary(probe)` prints the evidence behind the verdict and the recommendation:

```
summary(probe)
#>   Structural probe: full report
#> • System: 110,849 equations after condensation (68 backsolved variables;
#>   3,667,721 uncondensed)
#>   Structural rank: 110,849 of 110,849
#>   Rank at base data: 110,849 of 110,849
#> • Structure: no time chain; block partition by "reg" (66 blocks, 497 border
#>   variables)
#> • Simultaneous components: 1,947 of 58,290 components have more than
#>   one element; the largest has 50,548 rows
#> • Largest component by equation: e_peb ×6,172, e_qes1 ×5,280, e_qo ×2,640,
#>   e_po ×2,640, e_qc ×2,640, e_qpa ×2,640, and 40 more
#> • Statements: 178 equation statements, 1,150 statement-variable incidences
#>   Heaviest statement-variable incidences
#> # A tibble: 10 × 4
#>    eq        var    weight  rows
#>    <chr>     <chr>   <int> <int>
#>  1 e_qms     tfm   8764800  2640
#>  2 e_qds     tfd   8764800  2640
#>  ...
#>   Condensation
#> • 68 backsolved variables remove 97% of the uncondensed system (3,667,721 to
#>   110,849 equations)
#>   "reg" partitions the condensed system into 66 blocks, but below 120,000
#>   condensed equations threaded "LU" beats "DBBD"
#>   Recommendation
```

```
#>   "LU", 1 task × 8 threads for this container (8 cores, 12.0 GB)
#> • Evidence: 110,849 equations (condensed), no time chain, partition reg (66
#>   blocks, border 20.5% of the system), assessed at 2 tasks
#> • Rationale: one rank; threads for the condensed factorization
#> • Memory: about 4.0 GB at 1 task, 33% of 12.0 GB (fits)
#> → `ems_solve(cmf_path, matrix_method = "LU", n_threads = 8)`
#>   Files: report '/path/to/run/out/probe/sol.probe.json', solver log
#>   '/path/to/run/out/probe/solver_out_probe.txt'
```

Section by section:

- **System.** The size of the system the solver factorizes and, for a condensed model, how many variables were backsolved out and how large the model is before condensation.
- **Structural rank** and **Rank at base data.** Two tests of the closure. The first looks only at which variables each equation mentions; the second also discards entries whose base-data value is zero (for example a trade flow that is zero in the database), which can leave a variable with nothing to determine it even though the model's structure is fine. Both should read "N of N"; a smaller rank is a singular system (see below).
- **Structure.** What the parallel methods have to work with. A *time chain* is the period-by-period structure of an intertemporal model, which `"SBBD"` splits into blocks; a *block partition* is a set (usually the regions) along which `"DBBD"` splits a static system. The border variables are the ones that link the blocks; the fewer, the better the split.
- **Simultaneous components.** How much of the system must be solved at once. Most equations in a CGE model can be solved one after another once a few central ones are known; a *component* is a group that cannot, and the largest component shows how much of the system is genuinely simultaneous. `plot(probe, type = "components")` draws them.
- **Statements** and **Heaviest incidences.** The model's equations as statements, and the pairs of equation statement and variable that put the most entries into the system. `plot(probe, type = "incidence")` draws the whole pattern. These lines are for understanding the model; they do not affect the recommendation.
- **Condensation.** What backsolving likely does to this deployment (see when backsolving pays); the verdict is kept in `probe$condense$verdict`. `"faster"`: a condensed system without a block partition is likely bound to `"LU"`, where condensation tends to speed the solve. `"lu_fine"`: a condensed system with a partition below about 120 thousand condensed equations, where threaded `"LU"` was the fastest measured on a laptop. `"slower"`: above that size, or with a time chain, the partition belongs to a parallel method and substitution works against it, so the report suggests trying a redeploy without `backsolve`. `"candidate"`: an uncondensed `"LU"`-bound system of a million equations or more, which may gain from `backsolve` in `ems_model()`.
- **Recommendation.** The method, tasks and threads, then the *evidence* (the size, the time chain and the partition it was judged on, at the task count it was judged for), the *rationale* (the measured rule that produced it), and the *memory* estimate of the run against the container's memory. Further lines appear when they matter: `"DBBD"` would have been faster but does not fit; `"NDBBD"` replaces `"SBBD"` for the same reason; `"LU"` is excluded because its workspace would exceed the 32-bit limit; the `"DBBD"` refinement step is off. With `solution_method = "Johansen"` the method can differ, and a line says which it would be.

13.5.1.3. Singular systems

A structurally singular system prints differently:

```
#>   Structural probe                      10,530 equations
#>  Structurally singular: rank 10,527 of 10,530 (3 unmatched equations, 3
#>   unmatched variables)
#>   Undetermined: dprobeb ×3
#>   Over-constrained: e_dprobe2 ×3
#>  Locate it with `plot(x, type = "dm")`
#>  No solve recommendation: the closure is structurally singular; fix it first
```

Undetermined names the variables (with the number of elements) that no equation pins down: the closure leaves them endogenous but nothing determines them, so one of them should be exogenous, or an equation is missing. *Over-constrained* names the equations that compete for the same variables: too many equations for the variables they involve, so one of them should be dropped or one of its variables made endogenous. The two lists are two sides of the same defect, and swapping a variable between them is usually the fix. `Singular at base data` instead of `Structurally singular` means the structure is sound but zero flows in the data leave elements undetermined; the remedy is in the data or the aggregation, not the closure. `summary(probe)` lists every defective element, `probe$defects` holds them as a tibble, and `plot(probe, type = "dm")` draws where in the system they sit.

13.5.1.4. Files the probe leaves

The probe writes into `out/probe/` under the run directory, and overwrites it on every call: `sol.probe.json` (the report the object is built from), `sol.stats.json` (the measured structure) and `solver_out_probe.txt`, the solver's own log of the run. `probe$paths` has the paths. A probe never touches a solution already in the run directory.

The log is worth a look when the verdict surprises you. After the solver's banner and the setup timings, its `probe:` lines hold the whole diagnosis in order: the time chain found (intertemporal models), the partition candidates, one line per set with its block count, border size and whether it is usable, the set selected and its border, the Jacobian assembly, the structural and base-data ranks with the verdict in words, the simultaneous components and the largest one, the report path, and the total time. Lines starting with `Warning:` are the solver's warnings about the model file and are repeated on the console.

13.5.1.5. The rules behind the recommendation

Measured on a 12 GB laptop profile with 4 and 8 cores and on a 32-core server, they come down to the following; other models and machines may behave differently:

- **Intertemporal models:** `"SBBD"`, with up to 8 tasks and the remaining cores as threads. `"NDBBD"` only when `"SBBD"` would not fit in memory.

- **Static models:** `"DBBD"` at two tasks for an uncondensed system with a usable partition, more where cores and memory allow. Condensed systems, including the vetted GTAP models, suit `"LU"` with threads up to about 120 thousand condensed equations; above that the probe suggests trying a redeploy without backsolve for a parallel method. Uncondensed `"LU"` runs on one thread, since threads do not help it.
- **Small systems:** below 300 thousand equations (counting the backsolved ones), the smallest size measured, every method finishes in seconds and `"LU"` is recommended.
- **Tasks before threads:** more tasks usually beat more threads on the same cores, with little gain beyond 8 tasks.

For example, a 16-period GTAP-RE at 23 regions and 5 sectors on an 8-core laptop:

```
probe <- ems_probe(cmf_path, cores = 8, memory = 12)
#>   Structural probe                      616,509 equations
#>   Closure valid: every variable is determined and no equation is redundant,
#>   structurally and at base data
#> → Recommended for the machine given (8 cores, 12.0 GB): "SBBD", 8 tasks × 1
#>   thread
#>   `ems_solve(cmf_path, matrix_method = "SBBD", n_tasks = 8)`
#>   `summary(x)` for the full report

outputs <- ems_solve(
  cmf_path = cmf_path,
  solution_method = "Gragg",
  matrix_method = "SBBD",
  n_tasks = 8
)
```

13.6. Memory

Before a run starts, teems-R estimates its peak memory and compares it with the memory available to the container (on Docker Desktop, the limit under Settings > Resources). A run that clearly will not fit is refused before it starts, and the message names the remedies:

- a coarser aggregation or a condensed model,
- fewer tasks (each `"DBBD"` task holds its own copy of the data),
- `"NDBBD"` for an intertemporal model,
- more memory for Docker, or a larger machine.

A run close to the limit gets a warning instead. With `"DBBD"`, each linear solve also gets one refinement step, which holds the system matrix through the solve; the memory estimate includes it (see the `refine` package option). During the solve, the log reports memory after each phase, and the peaks are recorded in `model_diagnostics.txt`.

The solver sizes its factorization workspace automatically and grows it when needed, so `laA`, `laD` and `laDi` are normally left unset. If the log shows the workspace growing, passing the size recorded as `la_used` in `sol.stats.json` skips that search on the next run. A single `"LU"` factorization cannot

exceed the 32-bit workspace limit; a system that needs more must be condensed, aggregated more coarsely, or solved with a parallel method.

13.7. Checking solve quality

If a model solves but the results look implausible, the system may be numerically near-singular, typically because the closure depends on base-data flows close to zero. `condest = TRUE` measures how sensitive each linear solve is to the data:

```
outputs <- ems_solve(
  cmf_path = cmf_path,
  solution_method = "Gragg",
  matrix_method = "LU",
  condest = TRUE
)
```

Values of `kappa_w2` around `1e8` are normal for CGE models; values approaching `1e15` trigger a warning that the solution is unreliable. Look for near-zero flows the closure depends on, or rerun with `precision = "double"`. The check does not change results, costs a few percent of solve time, and applies to `"LU"` only.

`jacdump = TRUE` writes the linearized system at the base data, every equation element against every variable element, to `sol.jac` beside the solution, with the equation map in `sol.jac.json`. It is taken before the first step and does not change results.

13.8. Reproducibility

The same deployment solved with the same image gives byte-identical results with `"LU"`, `"SBBD"` and `"NDBBD"`, at any thread count and on any machine the image supports. Three cases agree only to rounding:

- `"DBBD"` with more than one thread, where a multi-step solve of a poorly conditioned model can amplify the rounding on its least determined results; use `n_threads = 1L` when byte-for-byte results matter,
- runs with more than one task, compared across machines,
- different image variants (for example `amd64` and `arm64`).

Each run records the solver version and numerical kernels it used in `model_diagnostics.txt`.

13.9. Solver version

Before every solve, teems-R checks that the image's solver version matches the package. A missing, old or mismatched solver stops the solve and names the pull or rebuild step. If the image still does not accept a solver option or input file that teems sends, the solve stops and names it rather than running without it; use the image released with your version of teems.

13.10. Concurrent runs

Several runs can solve at the same time, one deploy directory each. The directory is set by the `tempdir` option, so give every run its own before deploying it:

```
library(parallel)
outputs <- mclapply(c("Johansen", "Gragg"), function(method) {
  ems_option_set(tempdir = file.path("~/runs", method))
  dat <- ems_data(dat_input, par_input, set_input,
                  REG = "big3", ACTS = "macro_sector")
  cmf <- ems_deploy(dat, model, shock)
  ems_solve(cmf, solution_method = method)
}, mc.cores = 2)
```

`mclapply()` forks the R session, so each run sets its own `tempdir`; forking is not available on Windows, where `parLapply()` serves instead.

A second solve in a deploy directory that is already solving is refused. If an interrupted run leaves its claim behind, the message names the `.solve_lock` directory to remove.

13.11. Terminal mode

For long-running models, `terminal_run = TRUE` prepares the run and exits, so you can close R and launch the solve from a terminal:

```
ems_solve(
  cmf_path = cmf_path,
  solution_method = "Gragg",
  terminal_run = TRUE
)
```

13.12. Solving in-situ

`solve_in_situ()` calls the teems-solver directly with user-supplied input files, bypassing the `ems_data()` / `ems_model()` / `ems_deploy()` pipeline. The model file is parsed, but input files must be provided in their final form: no checks or modifications are applied to them. This suits pre-prepared input files, or re-solving the files written to disk by `ems_deploy()`.

```
solve_in_situ(
  ...,
  model_dir,
  model_file,
  closure_file,
  shock_file,
```

```
  ignore_condense = FALSE,
  solution_method = c("Gragg", "Johansen", "Euler", "Midpoint", "RK2",
                      "Heun", "RK4", "BoSha32", "DoPri54"),
  matrix_method = c("LU", "DBBD", "SBBD", "NDBBD"),
  n_subintervals = 1L,
  steps = NULL,
  n_tasks = 1L,
  n_threads = 1L,
  precision = c("single", "double"),
  verbosity = 1L,
  suppress_outputs = FALSE,
  terminal_run = FALSE,
  accuracy_vars = NULL,
  solver_args = NULL
)
```

13.12.1. Arguments

Argument	Default	Description
`...`		Input files, named after the model's `File` statements
`model_dir`		Directory the run uses; input files not already there are copied in, and outputs are written there
`model_file`		Path to the model Tablo (.tab) file
`closure_file`		Path to the closure (.cls) file
`shock_file`		Path to the shock (.shf) file
`ignore_condense`	`FALSE`	Set to `TRUE` when the closure and shock files come from a model loaded with `ems_model(..., ignore_condense = TRUE)`
`suppress_outputs`	`FALSE`	When `TRUE`, returns the CMF file path instead of parsed outputs
`solver_args`	`NULL`	Named list of everything `ems_solve()` takes through `...`, the Runge-Kutta controls included, e.g. `list(condest = TRUE)` or `list(adaptive = "yes")`

The solve arguments (`solution_method`, `matrix_method`, `n_subintervals`, `steps`, `n_tasks`, `n_threads`, `precision`, `accuracy_vars` and the rest) work as in `ems_solve()`.

13.12.2. Input files

Each name passed through `...` must match a `File` statement in the model Tablo file. For GTAP-RE, the Tablo file declares:

```
File GTAPDATA # base data file;
File GTAPINT  # intertemporal data file;
File GTAPPARM # parameter file;
File GTAPSETS # set file;
```

These names are then used as the named arguments:

```
solve_in_situ(
  GTAPDATA = file.path(write_dir, "teems", "GTAPDATA.txt"),
  GTAPINT = file.path(write_dir, "teems", "GTAPINT.txt"),
  GTAPPARM = file.path(write_dir, "teems", "GTAPPARM.txt"),
  GTAPSETS = file.path(write_dir, "teems", "GTAPSETS.txt"),
  model_dir = "~/in_situ_run/",
  model_file = "GTAP-RE.tab",
  closure_file = "GTAP-RE.cls",
  shock_file = "a_shock_file.shf",
  solution_method = "Gragg",
  matrix_method = "SBBD",
  n_tasks = 4L
)
```

The model file is processed exactly as `ems_model()` processes it, so in-TAB `Substitute` and `Backsolve` statements apply.

By default, `solve_in_situ()` returns the model's variables and coefficients; with `suppress_outputs = TRUE` it returns the CMF path, which can be passed to `ems_compose()`.

14. Composing results

14.1. Overview

`ems_compose()` retrieves and processes results from a solved model run. Results are parsed and filtered according to the `which` argument. Data validation and consistency checks are performed during the parsing process.

By default, `ems_solve()` calls `ems_compose()` automatically, so this function is primarily used when `suppress_outputs = TRUE` was passed to `ems_solve()` or `solve_in_situ()`, or when re-parsing results from a previous run.

```
ems_compose(
  cmf_path,
  which = "all",
  passes = FALSE
)
```

14.2. Arguments

Argument	Default	Description
`cmf_path`		Path to the CMF file generated by `ems_deploy()`
`which`	`"all"`	`"all"` returns every model variable and coefficient (coefficients at their updated, post-simulation values), except the condensation coefficients (`CSUB<n>`, see Condensation coefficients), which come back only when named. Otherwise, a character vector of variable and coefficient names to retrieve, matched without regard to case
`passes`	`FALSE`	`TRUE` adds the separate solutions of a three-pass `"Gragg"`, `"Midpoint"` or `"Euler"` run and their accuracy (see Separate solutions)

14.3. Basic usage

After a standard model run with suppressed outputs:

```
ems_solve(
  cmf_path = cmf_path,
  solution_method = "Gragg",
  matrix_method = "LU",
  suppress_outputs = TRUE
)

# Parse all results
outputs <- ems_compose(cmf_path = cmf_path)
```

14.4. Filtering output

Pass a character vector of names to `which` to retrieve a subset of results. Variables and coefficients can be mixed freely in a single call:

```
# All model variables and coefficients (default)
all_outputs <- ems_compose(cmf_path = cmf_path)

# Single variable by name
qfd <- ems_compose(cmf_path = cmf_path, which = "qfd")

# Single coefficient by name
SAVE <- ems_compose(cmf_path = cmf_path, which = "SAVE")

# Multiple names with mixed variables and coefficients
results <- ems_compose(cmf_path = cmf_path, which = c("qfd", "qfm", "SAVE"))
```

Note that some coefficient names differ from their associated HAR headers. For example, the coefficient exposed as `"VTMFSD"` corresponds to the `VTWR` header. Use the name as it appears in the model file, not the header.

14.5. Return value

`ems_compose()` returns a tibble with one row per result and four columns:

Column	Description
`name`	Variable or coefficient name as it appears in the model file
`label`	Description string from the model file

Column	Description
`type`	`"variable"`, `"coefficient"` or `"postsim"` (coefficients computed in a `PostSim` section)
`dat`	List-column of data.tables, one per row, containing the result values

14.6. Levels results

A variable declared with `ORIG_LEVEL=` in the model file has a known pre-simulation level: either a coefficient over exactly the variable's sets (GTAP's `qfd` takes `VDFB`, its value at base prices) or a number (`1.0` for a price index). Levels variables know theirs too. For these variables the data.table gains:

Column	Percent-change variable	Change variable
`pre_level`	pre-simulation level	pre-simulation level
`post_level`	`pre_level * (1 + Value / 100)`	`pre_level + Value`
`change` / `percent_change`	`change`: the ordinary change	`percent_change`: the percent change

For runs solved with `"BoSha32"` or `"DoPri54"` (see solving), each variable's data.table also has an `error_estimate` column. Sort it in descending order to find the least settled results; it is an indicator, not a bound on the error.

14.7. Separate solutions

A three-pass `"Gragg"`, `"Midpoint"` or `"Euler"` run extrapolates from three solutions with increasing step counts (see solving). `passes = TRUE` returns them, so the accuracy of each result can be inspected:

```
outputs <- ems_compose(cmf_path, passes = TRUE)
attr(outputs, "accuracy")
```

Each variable's data.table gains:

Column	Description
`pass1`, `pass2`, `pass3`	The solutions with the fewest to the most steps, in the units of `Value`
`figures`	Significant figures to which the two two-pass extrapolations agree (0 to 6; `NA` for a result judged zero, both extrapolations below 1e-6)

Column	Description
`levels_figures`	The same for the variable's levels value, which a percent-change result usually has more of
`error`	The difference between the two extrapolations, in the units of `Value`
`code`	The convergence code of GEMPACK manual 26.2.3, for example `"CX"` (confidence in the extrapolated result) or `"MD!"` (monotonic but appearing to diverge)
`summarized`	Whether the element counts in the run's accuracy summary: `TRUE` for endogenous and backsolved results (of the `accuracy_vars` variables when those are given), `FALSE` for exogenous elements, whose values the shock sets

The `accuracy` attribute has one row per variable: `elements`, `summarized`, the lowest figures (`min_figures`, `min_levels_figures`; a result judged zero counts as 6), the share of results at `accuracy_figures` or more (`share_figures`, `share_levels_figures`; see options), `worst_code` and `max_error`. With subintervals, `figures` and `levels_figures` are the lowest over the subintervals and `code` the worst; the pass columns and `error` come from the last subinterval alone, each pass compounded onto the extrapolated result of the subintervals before it. A run that wrote no separate solutions stops with an error.

Part V.

Configuration

15. Package options

15.1. Overview

teems provides a set of advanced options that control package behavior across function calls. For example, teems is rather chatty and users may wish to silence all innocuous messages with `ems_option_set(verbose = FALSE)`. Options are managed through three functions:

- `ems_option_get()` retrieves current option values
- `ems_option_set()` modifies option values
- `ems_option_reset()` restores all options to their defaults

```
ems_option_get(
  name = NULL
)

ems_option_set(
  tempdir = NULL,
  verbose = NULL,
  ndigits = NULL,
  accuracy_threshold = NULL,
  accuracy_figures = NULL,
  accuracy_criterion = NULL,
  required_figures = NULL,
  full_exclude = NULL,
  check_shock_status = NULL,
  timestep_header = NULL,
  n_timestep_header = NULL,
  docker_tag = NULL,
  assertions = NULL,
  range_test_initial = NULL,
  range_test_updated = NULL,
  random_seed = NULL,
  refine = NULL,
  converge = NULL
)

ems_option_reset()
```

Options persist for the duration of the R session and are reset on package reload.

15.2. Retrieving options

View all current options:

```
ems_option_get()
```

Retrieve a specific option by name. An error is raised if the name is not a valid option:

```
ems_option_get("verbose")
ems_option_get("ndigits")
```

15.3. Setting options

One or more options can be set in a single call:

```
ems_option_set(verbose = FALSE)
ems_option_set(verbose = FALSE, ndigits = 8)
```

15.4. Resetting options

Reset all options to their default values:

```
ems_option_reset()
```

15.5. Available options

Option	Default	Description
`tempdir`	`tempdir()`	Directory used for temporary file storage during a model run
`verbose`	`TRUE`	If `FALSE`, function-specific diagnostics are silenced
`ndigits`	`6`	Number of digits to the right of the decimal point kept when data are written to file. Values smaller than 0.1 in magnitude keep `ndigits` significant digits instead, so small nonzero data (e.g. `1e-10`) are not rounded to zero

Option	Default	Description
`accuracy_threshold`	`0.8`	Share of results that must be accurate to at least `accuracy_figures` significant figures. A warning is generated if the share is lower
`accuracy_figures`	`4`	Significant figures, from 1 to 6, a result must reach to count as accurate
`required_figures`	`NULL`	Figures, from 1 to 6, at which a result gets the convergence code `XRA` in the accuracy summary. Reporting only
`accuracy_criterion`	`"solution"`	The accuracy summary judged: `"solution"` the levels values of the endogenous results, `"data"` the values read from data files and updated, `"both"` the two together. A run that updates no data is judged on the solution
`full_exclude`	`c("DREL", "DVER", "XXCR", "XXCD", "XXCP", "SLUG", "EFLG")`	Headers to fully exclude from all aspects of the model run. Failure to designate these headers properly will result in errors. Modify with caution
`check_shock_status`	`TRUE`	If `FALSE`, no check on shock element endogenous/exogenous status is conducted
`timestep_header`	`"YEAR"`	Header written with the time of each period, as given in `time_steps` of `ems_data()` (years, or steps from the initial period); the model uses the differences between periods as interval lengths. For novel intertemporal models — modify with caution
`n_timestep_header`	`"NTSP"`	Header written with the number of periods (the length of `time_steps`). For novel intertemporal models — modify with caution
`docker_tag`	`"latest"`	Tag of the `teems` image to solve with
`assertions`	`"fatal"`	Severity of TAB `Assertion` failures during `ems_solve()`: `"fatal"`, `"warn"` or `"off"`
`range_test_initial`	`"fatal"`	Severity of declared-range violations (e.g. `(ge 0)`) on initial values, read or computed: `"fatal"`, `"warn"` or `"off"`
`range_test_updated`	`"warn"`	As `range_test_initial`, for updated values

Option	Default	Description
`random_seed`	`1`	Seed of the TAB function `RANDOM(a,b)`. The same seed gives the same draws on every run; each Formula element keeps its draw across solution steps. Change the seed for a new sequence (GEMPACK instead draws a new sequence per run by default)
`refine`	`TRUE`	One refinement step after every `"DBBD"` linear solve, which brings its accuracy to that of `"LU"`. It holds the system matrix through each solve, and the memory estimate counts it; `FALSE` turns it off. `"LU"` and `"SBBD"` are in practice as accurate as a single LU solve and are not refined; `"NDBBD"` is not refined, as it cannot yet keep its factorization for a second solve
`converge`	`FALSE`	`TRUE` applies GEMPACK's treatment of poorly converging results to runs extrapolated from three solutions: the result with the most steps where the three do not converge cleanly, their average where they are nearly equal. Not available with subtotals

A. Solver benchmarks

This appendix summarises the measurements behind the method recommendations in Solving the model. They show how solve time and memory change with model size and matrix method, first on a laptop-sized machine and then for models of up to 284 million equations on a server. The block methods follow Van Ha and Kompas (2016), who introduced `"SBBD"` on an intertemporal regional model of up to 32.7 million variables, Van Ha et al. (2017), who applied it to an intertemporal GTAP model of 108.5 million equations, and Kompas and Van Ha (2019), whose nested method solved an intertemporal GTAP model of 112 regions, 57 commodities and 47 periods, more than 500 million equations; see References.

A.1. How the runs were made

- **Models:** GTAPv7 (static) and GTAP-RE (intertemporal) on the GTAP 12 database, at the aggregations listed in each table, with all endowments. Static models were run both as the full system (`ignore_condense = TRUE`) and condensed (the vetted default).
- **Shock and method:** `aoall` +20 % everywhere, solved with Gragg 2-4-8.
- **Machine:** a 12 GB memory limit with 4 or 8 cores, emulating a 16 GB laptop running Docker Desktop. The tables show the 8-core runs, with resources written as tasks × threads. The large models ran on the full server: 32 cores and 125 GB.
- **Reading the times:** the runs used server processors, so absolute times will differ on your machine; the ratios between methods carry over. Each configuration ran once.
- **Solver:** 1.1.0 development builds, September 2026.

A.2. Intertemporal models

GTAP-RE, 8 cores; aggregations are REG × ACTS × periods. Resources are written tasks × threads, and "Best SBBD" is the fastest split of the 8 cores.

Aggregation	Equations	LU	SBBD 1 × 8	Best SBBD	Memory
big3 × full × 6	1.30M	177 s	52 s	34 s (8 × 1)	0.9 GB
big3 × full × 11	2.38M	435 s	96 s	60 s (8 × 1)	1.5 GB
big3 × full × 21	4.54M	–	189 s	106 s (4 × 2)	2.1 GB
WB7 × full × 11	5.87M	–	334 s	173 s (8 × 1)	3.2 GB
big3 × full × 41	8.85M	–	373 s	215 s (8 × 1)	4.5 GB

- SBBD is 5 to 7 times faster than LU.
- Tasks beat threads: splitting 8 cores into 4 or 8 tasks is 35 to 50 % faster than 1 task with 8 threads, and 8 tasks gain at most about 10 % over 4.

- Memory is modest; every intertemporal run fitted comfortably in 12 GB.

A.3. Static models, full system

GTAPv7 with `ignore_condense = TRUE`, 8 cores; aggregations are REG × ACTS. LU uses 8 threads, and "Best DBBD" is the fastest split of the 8 cores.

Aggregation	Equations	LU	Best DBBD	DBBD / LU
WB23 × energy	346k	36 s	17 s (4 × 2)	0.46
WB7 × full	534k	60 s	29 s (4 × 2)	0.48
R32 × medium	1.41M	470 s	170 s (8 × 1)	0.36
R32 × full	3.46M	2,618 s	564 s (4 × 2)	0.22
medium × medium	3.77M	3,095 s	789 s (2 × 4)	0.26
large × medium	7.69M	not finished in 3 h	2,373 s (2 × 4)	–

- DBBD is faster than LU at every size, and the advantage grows with size.
- LU gains almost nothing from threads on the full system.

A.4. Static models, condensed

The same aggregations with the vetted condensation, 8 cores. Condensation cuts the system about twentyfold.

Aggregation	Equations	LU 1 × 8	DBBD 2 × 4
WB23 × energy	20k	14 s	23 s
WB7 × full	25k	31 s	60 s
R32 × medium	70k	125 s	153 s
R32 × full	110k	456 s	511 s
medium × medium	140k	713 s	623 s
large × medium	230k	not finished in 3 h	out of memory

- Threaded LU is the faster choice up to about 100 thousand condensed equations; DBBD takes over above that.
- Condensed DBBD gets slower with more tasks, and needs more memory than on the full system.
- The largest model solves faster uncondensed with DBBD (2,373 s above) than condensed with either method.

A.5. Large models

A server with 32 cores (AMD EPYC 7763) and 125 GB of memory, with no container limit. All runs used Johansen except the first, which used Gragg 2-4-8. GTAP-INT is a research model that is not shipped with teems.

Model	Aggregation	Equations	Method	Time	Memory
GTAP-RE	WB7 × full × 41	21.9M	SBBD 8 × 8	422 s	10 GB
GTAPv7 (full)	full × full	40.5M	DBBD 8 × 1	1,449 s	87 GB
GTAP-RE	WB7 × full × 430	230M	SBBD 8 × 4	417 s	91 GB
GTAP-RE	WB7 × full × 468	250M	SBBD 8 × 1	690 s	100 GB
GTAP-INT	–	234M	NDBBD 1 × 32	13,715 s	130 GB
GTAP-INT	–	284M	NDBBD 1 × 8	21,135 s	132 GB

- SBBD solves intertemporal models of 250 million equations in minutes, as long as they fit in memory.
- For the same 40.5M-equation GTAPv7 system, an earlier build took 27 hours with LU on the full system and 2.9 hours with LU on the condensed model, against 24 minutes with DBBD above.
- NDBBD is much slower than SBBD, but it solves intertemporal models that SBBD cannot fit: on the 234M-equation GTAP-INT model, SBBD ran out of memory. The largest model solved, at 284M equations, used nearly all of the server's memory.
- Adding tasks raises memory: DBBD on 16 tasks ran out of memory on the 40.5M-equation system that 8 tasks solved.

A.6. Memory

Peak memory grows with the size of the *full* system, even when the model is condensed, because the data arrays follow the full model.

Method	Peak memory per million equations of the full system
LU	0.6 to 0.9 GB
DBBD	0.85 GB + 0.27 GB per task
SBBD	0.36 GB + 0.02 GB per task

A condensed model needs about the same memory as its full system under LU, and 1.3 to 1.55 times as much under DBBD.

Within 12 GB:

- intertemporal models fitted at every size measured (up to 8.85M equations; the model above suggests about 20M);
- full static systems fitted to 7.7M equations with LU or DBBD on 2 tasks; DBBD on 4 or more tasks ran out of memory at that size;
- condensed static systems fitted at every size with LU; DBBD on 8 tasks ran out of memory from 110 thousand condensed equations.

`ems_solve()` uses these estimates to stop a run that will not fit before it starts (see Memory).

B. References

The works below describe the methods that the teems solver and package implement, and the models they ship with.

B.1. Tablo language and solution procedure

Horridge J.M., Jerie M., Mustakinov D. & Schiffmann F. (2024), *GEMPACK manual* (documents Release 12.2), GEMPACK Software, Centre of Policy Studies, Victoria University, Melbourne, ISBN 978-1-921654-34-3. https://ideas.repec.org/p/cop/wpaper/gpman.html

Pearson K.R. (1991), 'Solving Nonlinear Economic Models Accurately via a Linear Representation', Impact Preliminary Working Paper No. IP-55, Melbourne (July; revised June 2002), pp.39. http://www.copsmodels.com/elecpapr/ip-55.htm

Dixon P.B., Parmenter B.R., Sutton J. & Vincent D.P. (1982), *ORANI: A Multisectoral Model of the Australian Economy*, North-Holland, Amsterdam.

B.2. Complementarities and subtotals

Harrison, W.J., Mark Horridge, K.R. Pearson and Glyn Wittwer (2004), 'A Practical Method for Explicitly Modeling Quotas and Other Complementarities', Computational Economics, vol. 23(4), pp. 325-341. [A preliminary version was Centre of Policy Studies and the Impact Project Preliminary Working Paper No. IP-78, Melbourne (April 2002), pp.19.] http://www.copsmodels.com/elecpapr/ip-78.htm

Harrison, W.J., J.M. Horridge and K.R. Pearson (2000), 'Decomposing Simulation Results with Respect to Exogenous Shocks', Computational Economics, vol.15, pp.227-249. [A preliminary version was Centre of Policy Studies and Impact Project Preliminary Working Paper No. IP-73, May 1999.] http://www.copsmodels.com/elecpapr/ip-73.htm

B.3. Runge-Kutta methods

Schiffmann, F. (2022), 'Runge Kutta integrators for fast and accurate solutions in GEMPACK', GEMPACK Software, Centre of Policy Studies, Victoria University, Melbourne (14 April).

Butcher, J.C. (2015), 'Runge-Kutta Methods for Ordinary Differential Equations', in M. Al-Baali, L. Grandinetti and A. Purnama (eds), *Numerical Analysis and Optimization*, Springer Proceedings in Mathematics & Statistics, vol. 134, pp. 37-58.

Bogacki, P. and L.F. Shampine (1989), 'A 3(2) pair of Runge-Kutta formulas', Applied Mathematics Letters, vol. 2, pp. 321-325.

Dormand, J.R. and P.J. Prince (1980), 'A family of embedded Runge-Kutta formulae', Journal of Computational and Applied Mathematics, vol. 6, pp. 19-26.

Gustafsson, K. (1991), 'Control theoretic techniques for stepsize selection in explicit Runge-Kutta methods', ACM Transactions on Mathematical Software, vol. 17, pp. 533-554.

B.4. Parallel matrix methods

Van Ha, P. and T. Kompas (2016), 'Solving intertemporal CGE models in parallel using a singly bordered block diagonal ordering technique', Economic Modelling, vol. 52, pp. 3-12. https://doi.org/10.1016/j.econmod.2015.07.011

Van Ha, P., T. Kompas, H.T.M. Nguyen and C.H. Long (2017), 'Building a better trade model to determine local effects: A regional and intertemporal GTAP model', Economic Modelling, vol. 67, pp. 102-113. https://doi.org/10.1016/j.econmod.2016.10.015

Kompas, T. and P. Van Ha (2019), 'The "curse of dimensionality" resolved: The effects of climate change and trade barriers in large dimensional modelling', Economic Modelling, vol. 80, pp. 103-110. https://doi.org/10.1016/j.econmod.2018.08.011

Horridge, J.M. and K.R. Pearson (2006), 'Running Simulations Faster on Multi-Processor or 64-Bit PCs via GEMPACK', Centre of Policy Studies Computing Document No. C15-01, Monash University (April).

B.5. Sparse linear algebra

Duff, I.S. and J.K. Reid (1993), 'MA48, a Fortran Code for Direct Solution of Sparse Unsymmetric Linear Systems of Equations', Rutherford Appleton Laboratory Report RAL-93-072, pp.62.

B.6. Software

HSL (2026), A collection of Fortran codes for large scale scientific computation. https://www.hsl.rl.ac.uk/

Balay, S. et al. (2026), *PETSc/TAO Users Manual*, Argonne National Laboratory, ANL-21/39 - Revision 3.25. https://doi.org/10.2172/2998643

Balay, S. et al. (2026), *PETSc Web page*. https://petsc.org/

B.7. Models

The vetted models are documented by their authors; each entry in teems-models lists its sources.

GTAPv6.2. Hertel, T.W. and M.E. Tsigas (1997), 'Structure of the Standard GTAP Model', chapter 2 in T.W. Hertel (ed), *Global Trade Analysis: Modeling and Applications*, Cambridge University Press. McDougall, R.A. (2000), 'A New Regional Household Demand System for GTAP', GTAP Technical Paper No. 20, Center for Global Trade Analysis, Purdue University.

GTAPv7 and the models built on it (GTAP-AEZ, GTAP-E, GTAP-EP, GTAP-RE). Corong, E.L., T.W. Hertel, R.A. McDougall, M.E. Tsigas and D. van der Mensbrugghe (2017), 'The Standard GTAP Model, Version 7', Journal of Global Economic Analysis, vol. 2(1), pp. 1-119. https://doi.org/10.21642/JGEA.020101AF

GTAP-AEZ. Hertel, T.W., H.-L. Lee, S. Rose and B. Sohngen (2009), 'Modeling Land-use Related Greenhouse Gas Sources and Sinks and their Mitigation Potential', chapter 6 in T.W. Hertel, S. Rose and R.S.J. Tol (eds), *Economic Analysis of Land Use in Global Climate Change Policy*, Routledge. Lee, H.-L., T.W. Hertel, B. Sohngen and N. Ramankutty (2005), 'Towards an Integrated Land Use Data Base for Assessing the Potential for Greenhouse Gas Mitigation', GTAP Technical Paper No. 25, Center for Global Trade Analysis, Purdue University. Baldos, U.L.C. and E.L. Corong (2020), 'Development of GTAP version 10 Land Use and Land Cover Data Base for years 2004, 2007, 2011 and 2014', GTAP Research Memorandum No. 36, Center for Global Trade Analysis, Purdue University.

GTAP-E. Corong, E.L. and A. Golub (2024), 'The GTAP-E Version 7 Model: An Energy-Environmental Version of the GTAP Model', Center for Global Trade Analysis, Purdue University. McDougall, R. and A. Golub (2007), 'GTAP-E: A Revised Energy-Environmental Version of the GTAP Model', GTAP Research Memorandum No. 15, Center for Global Trade Analysis, Purdue University. Burniaux, J.-M. and T.P. Truong (2002), 'GTAP-E: An Energy-Environmental Version of the GTAP Model', GTAP Technical Paper No. 16, Center for Global Trade Analysis, Purdue University.

GTAP-EP. The GTAP-E sources above, and Peters, J.C. (2016), 'GTAP-E-Power: An Electricity-detailed Economy-wide Model', Journal of Global Economic Analysis, vol. 1(2), pp. 156-187. Peters, J.C. (2016), 'The GTAP-Power Data Base: Disaggregating the Electricity Sector in the GTAP Data Base', Journal of Global Economic Analysis, vol. 1(1), pp. 209-250. https://doi.org/10.21642/JGEA.010104AF The shipped model follows the GTAP-E-Power model code, whose base-load and peak-load nests (`ELFEBL`, `ELFEPL`) are ordinary CES; the paper describes them as ACES.

GTAP-RE. Van Ha, P., T. Kompas and M. Cantele, 'Rethinking the Solution Strategy for Large Recursive CGE Models: Solving Recursive and Rational Expectations CGE Models the Non-Recursive Way', unpublished manuscript, University of Melbourne.

ORANI-G. Horridge, M. (2013), 'ORANI-G: A Generic Single-Country Computable General Equilibrium Model', Centre of Policy Studies and Impact Project, Monash University. Dixon, P.B., B.R. Parmenter, J. Sutton and D.P. Vincent (1982), *ORANI: A Multisectoral Model of the Australian Economy*, North-Holland, Amsterdam.